



COMSATS University Islamabad, Pakistan

Urdu Flow

A project presented to

COMSATS Institute of Information Technology, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Abdul Manan CIIT/SP22-BCS-011/SWL

Danish Nawaz CIIT/SP22-BCS-034/SWL

Hamid Zafar CIIT/SP22-BCS-041/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Abdul Manan

Danish Nawaz

Hamid Zafar

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “Urdu Flow” was developed by **Abdul Manan (CIIT/SP22-BCS-011)**, **Danish Nawaz (CIIT/SP22-BCS-034)** and **Hamid Zafar (CIIT/SP22-BCS-041)** under the supervision of “**Mr. Ali Sher Kashif**” and co supervisor “**Dr. Javed Ferzund**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

Mr. Ali Sher Kashif

Lecturer,

Department of computer science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Prof. Dr. Imran Sarwar Bajwa

Department of Computer Science

The Islamia University of Bahawalpur, Pakistan

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor,

Department of computer science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

Urdu Flow is an AI-powered bilingual language processing mobile application designed to streamline communication between Urdu and English speakers. The app integrates speech recognition, translation, transcription, and summarization into a single unified platform, offering users an efficient way to process text, audio, and documents. Built with Flutter and supported by NLP and Machine Learning technologies, it provides fast, accurate, and accessible language assistance for students, educators, content creators, and professionals.

The system leverages advanced tools such as Whisper-based speech-to-text, Google Translate API, transformer-based summarization models, and OCR for document extraction, ensuring high accuracy even with regional accents and varied input formats. Its four-tier architecture comprising the presentation, business logic, integration, and data layers enables scalability, modular development, secure authentication, and efficient data management through Firebase services. Users can seamlessly record audio, upload files, translate text, and generate summaries through a responsive and intuitive mobile interface.

In addition to user-focused features, Urdu Flow offers an admin panel for managing user accounts, configuring security settings, integrating APIs, and monitoring overall performance. By consolidating multiple language-processing capabilities into one application, Urdu Flow enhances productivity, reduces reliance on separate tools, and promotes accessible bilingual communication. The project represents a meaningful step toward improving Urdu-English language processing through innovative, secure, and AI-driven mobile technology.

Acknowledgment

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Mr. Ali Sher Kashif**” and our Co-Supervisor “**Dr. Javed Ferzund**”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Abdul Manan

Danish Nawaz

Hamid Zafar

Abbreviations

SRS	Software Requirements Specification
SDD	Software Design Description
iOS	iPhone Operating System
AI	Artificial Intelligence
ML	Machine Learning
UI	User Interface
UX	User Experience
API	Application Programming Interface
NLP	Natural Language Processing
TTS	Text-to-Speech
SDK	Software Development Kit
APK	Android Package Kit
DBMS	Database Management System
UC	Use Case
OCR	Optical Character Recognition
FR	Functional Requirement

Table of Contents

1.	Introduction.....	1
1.1.	Brief Overview.....	1
1.2.	Relevance to Course Modules	2
1.2.1.	Technical Skills	2
1.2.2.	Software Engineering Principles.....	2
1.2.3.	Database Design.....	2
1.2.4.	System Architecture	2
1.2.5.	UI/UX Design	3
1.3.	Project Background.....	3
1.4.	Literature Review.....	3
1.5.	Analysis from Literature Review	4
1.5.1.	Speech-to-Text & Low-Resource Urdu ASR.....	5
1.5.2.	Machine Translation Urdu To English	5
1.5.3.	OCR and Document Processing of Urdu Text	5
1.5.4.	UI/UX and Bilingual Content Handling	6
1.5.5.	Authentication, Security & Low-Resource Deployment	6
1.5.6.	Gaps and Suggested Extensions.....	6
1.6.	Methodology and Software Lifecycle.....	6
1.7.	Rationale behind Selected Methodology	7
1.8.	Summary	7
2.	Problem Definition.....	8
2.1.	Problem Statement.....	8
2.2.	Deliverables Requirements	8
2.2.1.	User Interface (UI) Design.....	9
2.2.2.	User Registration and Authentication	9
2.2.3.	Speech-to-Text Module.....	9
2.2.4.	Translation Module	9
2.2.5.	Transcription Module.....	9
2.2.6.	Summarization Module.....	9
2.2.7.	File Handling and Storage	9
2.2.8.	Search and Retrieval	9

2.2.9.	Security and Privacy	10
2.2.10.	Scalability	10
2.2.11.	Mobile App Development.....	10
2.2.12.	Documentation.....	10
2.2.13.	Testing and Quality Assurance.....	10
2.2.14.	User Support and Help Guides	10
2.2.15.	Regular Updates and Maintenance	10
2.3.	Current System.....	11
3.	Requirement Analysis	12
3.1.	Use Case Diagram.....	12
3.1.1.	Use Case Descriptions	13
3.2.	Functional Requirements	22
3.2.1.	User Interface Module	23
3.2.2.	Admin Modules	23
3.3.	Non-Functional Requirements	24
3.3.1.	Usability.....	24
3.3.2.	Performance	24
3.3.3.	Security	25
3.3.4.	Portability.....	25
3.3.5.	Reliability.....	25
4.	Design and Architecture.....	26
4.1.	System Architecture	26
4.2.	Data Representation	27
4.3.	Process Flow	27
4.3.1.	User Registration and Login	28
4.3.2.	Using the Main Modules.....	28
4.3.3.	Input Handling	29
4.3.4.	Processing the Request	29
4.3.5.	Displaying the Output.....	29
4.3.6.	Admin Panel Operations	29
4.3.7.	Logging Out.....	29
4.4.	Sequence Diagram	29
4.5.	Class Diagram.....	35

4.5.1.	Class Diagram for Urdu Flow	35
4.6.	Summary	36
5.	Implementation	37
5.1.	Algorithm.....	37
5.1.1.	User Authentication	37
5.1.2.	Speech-to-Text (Transcription) Algorithm.....	37
5.1.3.	Translation Algorithm	38
5.1.4.	Summarization Algorithm.....	38
5.1.5.	Chat Interaction.....	39
5.1.6.	Process & Convert Files Algorithm	39
5.2.	User Interface.....	40
6.	Testing and Evaluation.....	50
6.1.	Manual Testing.....	50
6.2.	System Testing	50
6.3.	Unit Testing.....	50
6.4.	Functional Testing.....	52
6.5.	Integration Testing.....	54
6.6.	Tools and Technologies.....	57
6.7.	Summary	57
7.	Conclusion	59
7.1.	Future Work	59
7.1.1.	Offline Processing Capabilities.....	59
7.1.2.	Advanced AI and Model Fine-Tuning	60
7.1.3.	Web-Based Admin and User Dashboard.....	60
7.1.4.	Enhanced OCR and Document Handling	60
	Reference	61

List of Tables

Table 1.1: Literature Review.....	4
Table 3.1: Login.....	13
Table 3.2: Sign up	13
Table 3.3: Upload Files.....	14
Table 3.4: Convert Speech	15
Table 3.5: Transcribe Audio Files.....	16
Table 3.6: Translate Content	16
Table 3.7: Manage User Profile	17
Table 3.8: Manage Users	17
Table 3.9: Summarize Content.....	18
Table 3.10: Process and Convert Files.....	19
Table 3.11: Download and Export Data.....	19
Table 3.12: Provide Feedback.....	20
Table 3.13: Configure Security & Settings	20
Table 3.14: Integrate Third-Party APIs.....	21
Table 3.15: View Feedback	21
Table 3.16: Monitor Performance	22
Table 5.1: External API's	40
Table 6.1: Login as an Admin.....	51
Table 6.2: Login as an existing user.....	51
Table 6.3: Logout as an Admin.....	52
Table 6.4: Logout as User	52
Table 6.5: Login as Admin	53
Table 6.6: Summary of Admin Feature.....	54

List of Figures

Figure 3.1: Use case diagram of Urdu Flow	12
Figure 4.1: Architecture Diagram	26
Figure 4.2: Process Flow Diagram.....	27
Figure 4.3: Process Flow Diagram.....	28
Figure 4.4: Sequence Diagram of Urdu Flow Authentication Module.....	30
Figure 4.5: Sequence Diagram of Urdu Flow Summarization Module	31
Figure 4.6: Sequence Diagram of Urdu Flow Transcription Module	32
Figure 4.7: Sequence Diagram of Urdu Flow Translation Module	33
Figure 4.8: Sequence Diagram of Urdu Flow File Management Module.....	34
Figure 4.9: Sequence Diagram of Admin Operations.....	35
Figure 5.1: UI for Login page of Urdu Flow	41
Figure 5.2: UI for Sign Up page of Urdu Flow.....	42
Figure 5.3: UI for Home page of Urdu Flow	43
Figure 5.4: UI for Edit Profile page of Urdu Flow	44
Figure 5.5: UI for Summarization page of Urdu Flow	45
Figure 5.6: UI for Translation page of Urdu Flow	46
Figure 5.7: UI for Convert to Speech Page of Urdu Flow	47
Figure 5.8: UI for Settings Page of Urdu Flow.....	48

CHAPTER 1

INTRODUCTION

1. Introduction

Urdu Flow is a mobile application developed to address the practical challenges faced by users who work across Urdu and English languages. Many students, educators, and professionals frequently need to convert speech to text, translate content, transcribe audio, or summarize documents, yet there is no unified platform that performs all these tasks accurately for Urdu. Existing tools offer limited Urdu processing, or require users to switch between multiple applications.

Urdu Flow provides a single, easy-to-use solution that integrates speech recognition, translation, transcription, and summarization. The app is designed specifically for Urdu and English, ensuring that users can process audio, text, and documents more reliably. By leveraging modern NLP and machine learning models, the system aims to improve accuracy for everyday and academic usage.

The application also addresses the difficulty users face when handling multilingual content. Whether converting spoken Urdu into English text, extracting text from scanned documents, or summarizing lengthy material, Urdu Flow streamlines these tasks through a simple interface. This makes the system accessible to users with varying levels of technical experience.

Currently, no dedicated mobile application offers this level of Urdu-focused language processing. Urdu Flow fills this gap by providing a consolidated and efficient tool that enhances productivity and supports clearer communication. Its straightforward design ensures that users can navigate features easily and obtain the required results without unnecessary complexity.

In summary, Urdu Flow is developed to support users in processing Urdu and English content more effectively. By combining key language-processing capabilities into one platform, the application aims to provide a practical and dependable solution for both academic and everyday needs.

1.1. Brief Overview

The Urdu Flow project aims to develop a mobile application that provides essential Urdu–English language processing features in one place. The app supports speech-to-text, text translation, audio/document transcription, and text summarization. It is being developed using Flutter for mobile platforms and Firebase as the backend to ensure secure storage, fast processing, and smooth user experience. The development follows an iterative approach to gradually refine each module.

Urdu Flow is designed to assist users who regularly work with Urdu and English content. Unlike general-purpose tools, it focuses specifically on handling Urdu accurately, especially where existing systems struggle with accents or formatting. By offering multiple processing features in a single application, Urdu Flow makes it easier for users to convert, translate, and summarize content without switching between different apps.

1.2. Relevance to Course Modules

The development of Urdu Flow directly applies multiple concepts and skills gained throughout the Computer Science degree.

1.2.1. Technical Skills

The project makes use of programming tools and technologies learned during the course, including Flutter for mobile development, Python for model handling, Firebase for backend services, and APIs for language processing. These skills were essential in implementing speech-to-text, translation, transcription, and summarization modules.

1.2.2. Software Engineering Principles

The project follows core software engineering practices such as requirement analysis, system modeling, design, implementation, and testing. Concepts from software process models, documentation, and modular development helped ensure that Urdu Flow remains scalable, maintainable, and structured.

1.2.3. Database Design

A strong understanding of database management was applied while designing Firebase Firestore collections for storing user data, session logs, uploaded files, and processed results. Proper data structuring enables smooth retrieval and supports efficient integration with the app's processing modules.

1.2.4. System Architecture

Understanding of mobile application architecture and cloud integration was necessary to build a multi-layered system. Concepts such as client-server architecture, API communication, and integration of third-party NLP services were used to design a reliable and organized system.

1.2.5. UI/UX Design

Principles from human-computer interaction and user interface design courses were applied to develop a simple and accessible interface. This ensures users can easily navigate features such as uploading files, recording audio, translating content, or viewing results.

1.3. Project Background

The need for the Urdu Flow project arises from several challenges faced by students, educators, and professionals who frequently work across Urdu and English languages. Before Urdu Flow, users had no single platform that could accurately perform Urdu speech-to-text conversion, translate between Urdu and English, transcribe audio or documents, and summarize content. Existing tools only cover part of these requirements and often struggle with regional accents, Urdu text formatting, or document processing, making them unreliable for academic and professional use [\[8\]](#).

Users commonly relied on multiple separate applications, one for speech recognition, another for translation, and others for transcription or summarization. This scattered workflow caused inefficiency, inconsistent output quality, and increased time consumption. Additionally, many widely used tools provide limited support for Urdu, resulting in inaccurate transcriptions, incorrect translations, and poor handling of colloquial or accent-specific variations.

To address these limitations, Urdu Flow was developed as an all-in-one language processing application that allows users to:

- Convert Urdu or English speech into accurate text.
- Translate text or transcriptions between Urdu and English.
- Transcribe audio files and scanned/taped documents.
- Generate concise summaries of long content.

By offering these features within a single mobile application, Urdu Flow streamlines multilingual tasks, improves accessibility, and provides a more dependable solution for users who depend on Urdu–English language processing in daily academic, personal, or professional work.

1.4. Literature Review

Recent advancements in Natural Language Processing (NLP) and Machine Learning (ML) have significantly improved multilingual communication technologies, particularly for high-

resource languages. However, research consistently highlights the challenges faced by low-resource languages such as Urdu, especially in speech recognition, machine translation, transcription accuracy, and summarization quality. Studies on Urdu speech recognition demonstrate progress using models like HMMs [1] and modern transformer-based architectures, but also emphasize persistent issues with acoustic variability, accent diversity, and limited training datasets, which restrict real-world performance and usability. Similarly, research in Urdu-English machine translation and transliteration suggests that neural models offer improved fluency and context handling, yet still struggle with domain-specific vocabulary, idiomatic expressions, and morphological complexity, making fully accurate bilingual communication challenging [4].

Several existing NLP utilities and applications have inspired this project.

Table 1.1: Literature Review

App	Weaknesses	Proposed Solution
Google Translate	Limited accuracy for colloquial Urdu phrases and regional accents.	Improved translation accuracy with fine-tuned models for Urdu-English and English-Urdu language pairs.
Microsoft Azure Speech Services	High cost, complex setup for Urdu processing, and limited real-time usability for general users.	Offers a simplified mobile interface using optimized speech-to-text processing specifically for Urdu and English.
Otter.ai	No support for Urdu transcription. English-only focus, cannot process Urdu documents or audio.	Supports Urdu and English transcription, including audio uploads and scanned documents.
Generic OCR/Document Tools	Poor accuracy for Urdu text, formatting errors, and lack of translation or summarization features.	Integrates OCR-based Urdu text extraction along with translation and summarization, ensuring full document processing.

1.5. Analysis from Literature Review

This section analyses the key findings from referenced literature in the project documents for the Urdu Flow application. It focuses on themes relevant to your system: speech-to-text (STT) for Urdu/English, translation of Urdu to English, OCR and document summarization, UI/UX for multilingual content, and authentication/security. The analysis highlights methodologies, performance metrics, and implications for the Urdu Flow design. Where content was limited (due to access restrictions or insufficient coverage) this has been noted, and alternative directions suggested. The aim is to substantiate our architectural and module-level design choices e.g., using fine-tuned translation models, integrating OCR plus summarization pipelines, and building a mobile-friendly bilingual UI.

1.5.1. Speech-to-Text & Low-Resource Urdu ASR

Research shows that automatic speech recognition (ASR) for Urdu remains challenging due to limited labeled corpora and complex phonology. For example, one study proposes an end-to-end neural network model for Urdu ASR, using semi-supervised learning to compensate for data scarcity. [1] The implication for Urdu Flow is that your STT module must account for low-resource conditions hence justifying design decisions to use pretrained models (e.g., Whisper or equivalent) then fine-tune or adapt for Urdu. It also argues for using confidence scores, noise robustness, and fallback mechanisms.

1.5.2. Machine Translation Urdu To English

Several studies highlight the limitations of machine translation (MT) systems for Urdu. A comprehensive evaluation of English-to-Urdu translation using multiple models (LLM, bilingual, multilingual) shows lower BLEU scores for Urdu language pairs compared to high-resource languages [10]. A separate study developed a fully convolutional NMT model for Urdu-English and reported improved BLEU scores compared to baseline models. [9] These findings validate your project’s focus on improving accuracy for colloquial Urdu. Therefore the design of Urdu Flow’s translation module using Google Translate API plus post-processing or fine-tuning is well grounded.

1.5.3. OCR and Document Processing of Urdu Text

Literature in document-image processing demonstrates that Urdu OCR performance lags behind English. One recent paper on deep-learning based OCR for Urdu reports large improvements in handwritten or noisy fonts but still notes formatting and segmentation errors. Given this, our architecture decision to combine OCR, translation, summarization for

document uploads is justified. It also suggests that you should plan for error- handling, layout correction and user feedback in your app.

1.5.4. UI/UX and Bilingual Content Handling

While there is less direct academic work specific to bilingual Urdu/English mobile UIs, the literature on multilingual UX emphasizes simplicity, language-toggle features, and localized text and audio support. These findings support your choice of a clean interface that requires minimal training, and bilingual support that extends beyond mere translation into context-aware presentation.

1.5.5. Authentication, Security & Low-Resource Deployment

In low-resource language applications, user authentication and data protection are often overlooked. Although direct studies on Urdu-language STT/translation apps are scarce, best-practice guidelines (from broader mobile-app literature) emphasize secure hashing of credentials, minimal permissions, offline fallback, and auditing. This underpins your decision to use secure authentication (e.g., hashed passwords, Firebase Auth) and to plan for future enhancements like 2FA and encryption.

1.5.6. Gaps and Suggested Extensions

- There is limited research specifically on mobile friendly bilingual (Urdu/English) summarization pipelines combining OCR, STT, translation, summarization. This gap presents an opportunity for innovation in our project.
- Few works evaluate real-time mobile transcription of Urdu audio with varying accents/environments.
- Most MT research focuses on text, translation of scanned documents, mixed media and colloquial speech remains under-explored your document upload, translation, summarization module can make a valuable contribution.
- The literature suggests leveraging multilingual synthetic datasets for low-resource languages [5] this could be an extension for future work.

1.6. Methodology and Software Lifecycle

Urdu Flow is a software project that includes multiple modules such as speech-to-text, translation, transcription, and summarization. Since these features require gradual refinement and testing, it is important to use a development methodology that supports continuous improvement and structured updates. The Iterative Model is well suited for this project because

it allows the system to be developed in controlled phases, where each cycle enhances the functionality based on testing and feedback.

1.7. Rationale behind Selected Methodology

The Iterative Model is a software development approach where the system is built step by step, with each iteration producing a working version of the application. This model is effective for Urdu Flow because language-processing features require repeated tuning, accuracy checks, and integration of third-party services like NLP models, OCR, and speech recognition.

The model also supports reviewing each module—such as transcription or translation—independently before moving to the next stage. This ensures that issues can be identified early, making development more manageable and improving overall reliability.

Key Principles of the Iterative Model:

- **Iterative Development:** The application is developed in cycles. Each iteration focuses on refining core modules like speech recognition, translation, or summarization.
- **Incremental Enhancement:** New features or improvements are added with every cycle. Early iterations implement basic functionality, while later iterations improve accuracy, UI, and performance.
- **Continuous Testing and Feedback:** Each iteration undergoes testing to evaluate the accuracy of NLP models, the performance of the app, and the correctness of data handling. Feedback is used to improve the next cycle.
- **Adaptive Planning:** The development plan is updated as new requirements emerge, such as integrating better NLP models, improving UI flows, or enhancing Firebase storage structure.

1.8. Summary

This chapter explains the project's purpose, its connection to course concepts, and insights from related research. It also describes the chosen method and why it fits the project.

CHAPTER 2

Problem Definition

2. Problem Definition

This chapter discuss the precise problem to be solved. It should extend to include the outcome.

2.1. Problem Statement

Users frequently face difficulties when working across Urdu and English due to the lack of a unified and accurate language-processing platform. Currently, most available tools offer only partial support such as translation without transcription, or speech-to-text without summarization. These fragmented solutions make it challenging for students, professionals, and educators to efficiently convert, translate, or summarize Urdu and English content.

Existing systems also struggle with Urdu-specific challenges, such as colloquial vocabulary, and low-quality text extraction from documents. As a result, users often receive inaccurate transcriptions or translations, forcing them to use multiple applications to complete a single task. This leads to inconsistencies, increased effort, and reduced productivity.

Furthermore, users have no structured way to process audio, documents, and text within one environment. Whether they need to transcribe a lecture, translate handwritten or scanned notes, or summarize long content, there is no dedicated mobile platform that supports all these needs with a focus on Urdu.

To address these limitations, Urdu Flow is being developed as an integrated mobile application that combines speech recognition, translation, transcription, and summarization in one system. The app allows users to convert audio to text, translate between Urdu and English, extract text from documents, and generate concise summaries all within a single interface. This ensures accuracy, convenience, and improved accessibility for everyday and academic tasks.

The absence of such a dedicated Urdu-focused language-processing platform clearly highlights the need for Urdu Flow, enabling users to handle bilingual content more efficiently and reliably.

2.2. Deliverables Requirements

The Urdu Flow project's deliverables and development requirements include all components necessary to build a functional and reliable Urdu–English language processing application. These are the key deliverables:

2.2.1. User Interface (UI) Design

Develop a simple and accessible interface that allows users to record audio, upload files, translate text, and view summaries. The UI must support both Urdu and English, ensuring smooth navigation for all user types.

2.2.2. User Registration and Authentication

Implement a secure login and registration system using Firebase Authentication. Users should be able to create accounts, manage their profiles, and access all modules securely.

2.2.3. Speech-to-Text Module

Provide functionality for converting Urdu or English speech into text using integrated speech recognition models. Support recording through the app and uploading audio files.

2.2.4. Translation Module

Enable accurate translation between Urdu and English. Integrate APIs and apply post-processing to improve translation quality for regional variations.

2.2.5. Transcription Module

Allow users to upload audio files or documents for transcription. Incorporate OCR for extracting Urdu text from images or scanned files.

2.2.6. Summarization Module

Generate concise summaries of long Urdu or English content. Support summarization for uploaded documents, typed text, and transcription outputs.

2.2.7. File Handling and Storage

Use Firebase Storage and Firestore to save uploaded files, processed results, session logs, and user data. Ensure organized and secure data management.

2.2.8. Search and Retrieval

Enable users to search their history, processed files, or saved outputs for easy access and reuse.

2.2.9. Security and Privacy

Implement secure authentication, encrypted data handling, and access control to protect user information and ensure compliance with data-security standards.

2.2.10. Scalability

Design the backend to support increasing numbers of users, larger files, and extended modules. Firebase's cloud-based services ensure long-term scalability.

2.2.11. Mobile App Development

Develop the application using Flutter for cross-platform support on Android and iOS. Integrate backend services, machine-learning models, and UI flows.

2.2.12. Documentation

Prepare user manuals, API documentation, and developer documentation to support both end-users and future developers of the system.

2.2.13. Testing and Quality Assurance

Perform unit testing, integration testing, and user acceptance testing to ensure that speech recognition, translation, transcription, and summarization work correctly across different scenarios.

2.2.14. User Support and Help Guides

Create tutorials and guides to help users understand how to record audio, upload files, translate content, and download results.

2.2.15. Regular Updates and Maintenance

Plan for updates based on user feedback, accuracy improvements to NLP models, and periodic performance and security enhancements.

2.3. Current System

At present, there is no single platform that accurately supports Urdu speech-to-text, translation, transcription, and summarization in one place. Users must rely on multiple separate tools, which leads to inconsistent results and a time-consuming workflow.

Most existing applications offer limited support for Urdu, especially for colloquial terms, and document text extraction. This results in inaccurate transcriptions, weak translations, and unreliable OCR output.

Due to the absence of an integrated Urdu-focused solution, users struggle to process audio, text, and documents efficiently, creating a clear need for a unified system like Urdu Flow.

CHAPTER 3

REQUIREMENT ANALYSIS

3. Requirement Analysis

The requirement analysis defines the expected behaviour, user interactions, and overall operational flow of the Urdu Flow bilingual language processing system. It outlines core functional requirements such as speech-to-text conversion, Urdu–English translation, audio and document transcription, and automated summarization, along with essential non-functional requirements covering performance, security, usability, and reliability. Use case models and diagrams help structure these requirements by illustrating how users and administrators interact with the system, ensuring that each feature aligns with real user needs and system constraints. This structured approach guides system design, supports efficient planning and resource allocation, and provides a strong foundation for testing to ensure Urdu Flow operates accurately and consistently.

3.1. Use Case Diagram

A use case diagram is like a friendly map that visually shows how users (actors) interact with a system, helping everyone easily understand its functions and requirements.

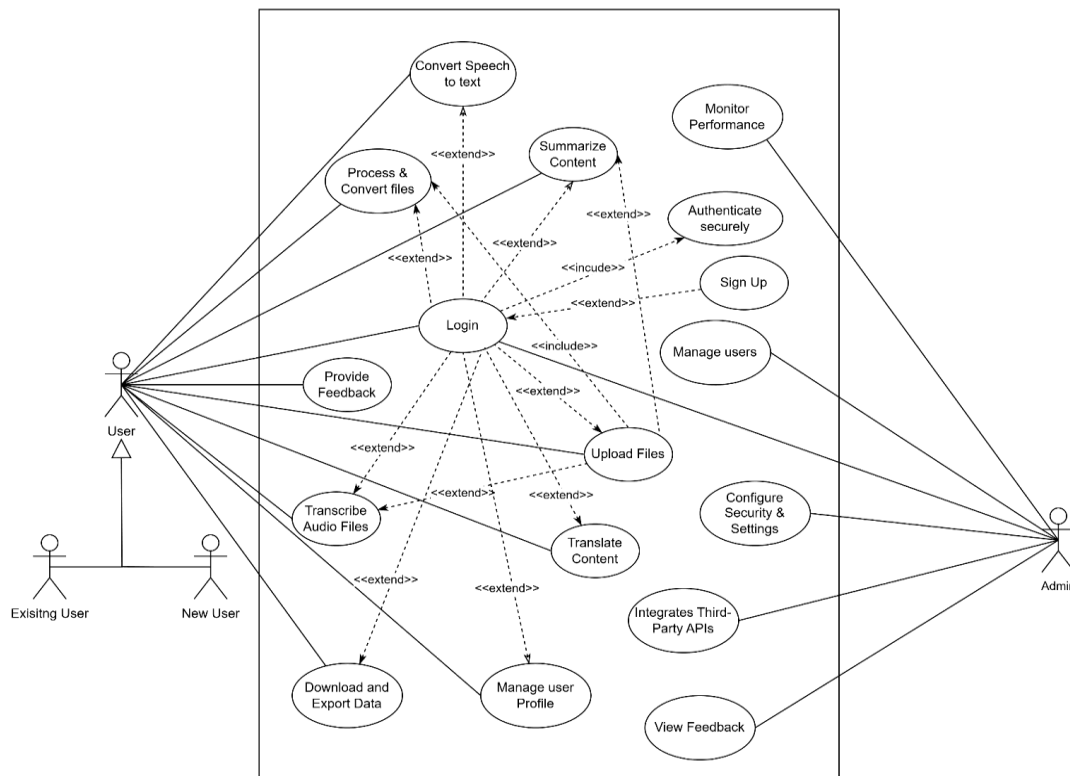


Figure 3.1: Use case diagram of Urdu Flow

3.1.1. Use Case Descriptions

A use case description details how users engage with Urdu Flow to accomplish tasks. For example, a user signs up, logs in, uploads an audio file, converts speech to text, and then downloads the resulting Urdu transcript content.

Table 3.2: Login

Use Case ID	UC-01
Use Case Name	Login
Actors	User Admin
Description	A user logs into the system.
Trigger	User requests to log in.
Pre-conditions	The user has an account.
Postconditions	The actor is now logged into the system.
Normal Flow	This use case starts when an actor tries to log in to the system. The system asks that the actor enter either an email or a phone number and a password. The actor inputs their email or phone number along with their password. The system validates the entered credentials and logs the actor in, making the process smooth and welcoming.
Alternative Flows	If you enter an incorrect email, phone number, or password, you'll see an error message guiding you to try again.
Exceptions	Incorrect credentials. Forget Password.
Business Rules	The system should be connected to the network to work smoothly.

Table 3.3: Sign up

Use Case ID	UC-02
Use Case Name	Sign Up
Actors	User
Description	A new user signs up for an account in the system.
Trigger	The user requests to sign up.

Pre-conditions	When the user doesn't have an account to log in to the system.
Postconditions	The user's details are safely stored in our database, ensuring your information is kept secure.
Normal Flow	The use case starts when the actor decides to sign up. The system then shows the signup page, inviting users to enter their username or email, first and last name, and create a password. The actor enters this information. The system reviews everything and creates a new account. The system happily confirms that the registration was successful, making the whole process smooth and welcoming.
Alternative Flows	Users can easily enter their information and submit it. If there's any invalid info, the system kindly prompts them to make corrections. If the email/phone number is already registered, an error message is displayed.
Exceptions	If the user already exists, or if the information is not in the correct format or missing, the system will kindly display an error message accordingly.
Business Rules	Active internet access is available.

Table 3.4: Upload Files

Use Case ID	UC-03
Use Case Name	Upload Files
Actors	User
Description	A user uploads a file for processing, transcription, or summary.
Trigger	The user selects a file to upload.
Pre-conditions	The user logged into the system.
Postconditions	The file is successfully uploaded.
Normal Flow	The use case begins when the user chooses the upload option. The system prompt user to ask for a file. The user then selects and uploads their file. The system checks the file type and securely stores it. The system confirms successful upload.
Alternative	If the file type isn't supported, you'll see an error message.

Flows	
Exceptions	Upload failed due to network issues. File size is too large.
Business Rules	The file format must be supported.

Table 3.5: Convert Speech

Use Case ID	UC-04
Use Case Name	Convert Speech
Actors	User
Description	The user converts spoken speech into text by either recording audio directly or uploading an audio file.
Trigger	The user initiates a request to transcribe speech into text.
Pre-conditions	Microphone access is enabled, or you've uploaded a valid audio file, and you're logged in.
Postconditions	The system generates a text output.
Normal Flow	The user chooses the "Convert Speech to Text" feature. The system then offers the option to record audio or upload an existing audio file. The user can either record speech via the microphone or upload a supported audio file. The system processes the audio and converts the spoken words into text. The transcribed text is then shown to the user. The user has the option to download, copy, or edit the transcribed text.
Alternative Flows	The user chooses not to continue until they submit audio input. If the request cannot be processed, an error message is displayed.
Exceptions	No Internet connection. Microphone access is denied. The chosen file format is invalid.
Business Rules	The system needs to support standard audio formats.

Table 3.6: Transcribe Audio Files

Use Case ID	UC-05
Use Case Name	Transcribe Audio Files
Actors	User
Description	The system transcribes uploaded audio files into text.
Trigger	The user requests transcription.
Pre-conditions	The user is logged in and has uploaded an audio file.
Postconditions	The system provides a text transcript.
Normal Flow	The user selects an audio file. The system processes the file. The system converts the audio to text. The transcribed text is displayed.
Alternative Flows	The user can request different options for formatting the text.
Exceptions	Poor audio quality can make transcription errors more likely, but with good quality recordings, things tend to go much smoother.
Business Rules	The system must be connected to the network.

Table 3.7: Translate Content

Use Case ID	UC-06
Use Case Name	Translate Content
Actors	User
Description	The system translates text into Urdu and English, making it easier for everyone to understand and communicate.
Trigger	The user begins the translation process by choosing the module.
Pre-Condition	The user is logged in and ready to translate text, which can be in Urdu or English.
Postconditions	The system offers translated text to help you understand content better. We're here to make sure language barriers are no longer a hurdle for you.
Normal Flow	The user inputs or uploads a text file for translation.

	Then, they click the translate button to initiate processing. The system processes the text, and the translated version is displayed.
Alternative Flows	The user can request various text formatting options. If the provided text is in a language not supported, an error message will be displayed.
Exceptions	Low-quality audio can cause transcription errors.
Business Rules	The system needs to be connected to the network to ensure everything runs smoothly and efficiently.

Table 3.8: Manage User Profile

Use case ID	UC_07
Use case name	Manage User Profile
Actors	User
Description	The user modifies their personal information in their profile.
Trigger	The user chooses the profile settings option.
Pre-condition	The user is logged in.
Postcondition	Profile details are updated.
Normal flow	The user chooses their profile settings. The system then shows current details. Afterward, the user makes updates. The system validates and saves the changes.
Alternative flow	An error message appears if an invalid email or phone number is entered.
Exceptions	The profile update couldn't be completed because of a system issue.
Business rules	Your email or phone number needs to be both valid and unique.

Table 3.9: Manage Users

Use case ID	UC_08
Use case name	Manage Users
Actors	Admin
Description	The administrator oversees system users.

Trigger	The administrator chooses the user management option.
Pre-condition	The administrator has successfully logged into the system.
Post-condition	User access information and details are kept current.
Normal flow	The administrator manages the user accounts. The system shows the list of users' information. The administrator can add, modify, or remove user profiles as needed. The system stores your modifications automatically.
Alternative flow	When a user account is deleted, their data is stored securely for a specific period.
Exceptions	The user's record cannot be accessed or completed.
Business rules	This feature is accessible only to admins.

Table 3.10: Summarize Content

Use case ID	UC_09
Use case name	Summarize Content
Actors	User
Description	This system can analyze and summarize texts or transcriptions that you upload, providing clear and human-like descriptions.
Trigger	The user asks for a summary of the content.
Pre-condition	The system has either transcribed the text or uploaded a document that is ready for review.
Postcondition	The system creates and shows a condensed summary of the content.
Normal flow	The user chooses a document or transcribed content for their use. The system analyzes the text and pulls out essential details. The system creates a brief summary of the content. The summarized content is shown.
Alternative flow	If the text is too brief to summarize, the system will indicate that there's no need for a summary.
Exceptions	Poorly formatted writing can cause incorrect results.
Business rules	Summarization is carried out using techniques that are based on artificial intelligence.

Table 3.11: Process and Convert Files

Use case ID	UC_10
Use case name	Process and Convert Files
Actors	User
Description	The system allows users to upload files, which it then processes and converts into various formats.
Trigger	The user chooses a file to process and convert.
Pre-condition	The user has uploaded a compatible file.
Postcondition	The file has been processed and transformed into the preferred format.
Normal flow	A user chooses a file for processing. The system examines the file type to understand it better. The user chooses the preferred output format. The system handles and transforms the data. The document has been saved successfully and is now prepared for use.
Alternative flow	If the file format isn't compatible, the system will alert the user.
Exceptions	Corrupt files cannot be processed.
Business rules	The system only allows specific types of files to be converted.

Table 3.12: Download and Export Data

Use case ID	UC_11
Use case name	Download and Export Data
Actors	User
Description	The user retrieves completed files or exported data.
Trigger	The user chooses the option to download or export.
Pre-condition	The user has uploaded files or created content.
Postcondition	The system supplies the file for your use.
Normal flow	The user chooses a file or report to download. The system handles the request. The system makes the file available for download. The individual obtains the file and stores it on their device.
Alternative flow	If the file is too big, the system recommends using a different compression

	format.
Exceptions	Download interrupted due to network failure.
Business rules	Only processed files or reports are available for export.

Table 3.13: Provide Feedback

Use case ID	UC_12
Use case name	Provide Feedback
Actors	User
Description	The user provides comments or opinions regarding the system.
Trigger	The user chooses the feedback.
Pre-condition	The user has successfully signed into their account.
Postcondition	The system retains the feedback.
Normal flow	The user chooses the feedback feature. The system provides a feedback form for users to share their opinions. The user completes the form and submits it. The system saves the feedback provided by users for further evaluation.
Alternative flow	If a user leaves required fields blank, the system will ask them to fill in the missing information.
Exceptions	Network issues are preventing feedback from being submitted.
Business rules	The administrator should go over the feedback.

Table 3.14: Configure Security & Settings

Use case ID	UC_13
Use case name	Configure Security & Settings
Actors	Admin
Description	The administrator sets up the security features and system preferences.
Trigger	The administrator sets up the security features and system preferences.
Pre-condition	The administrator is currently logged in.
Postcondition	The security configurations of the system have been modified.
Normal flow	The administrator chooses the security options and adjusts the settings. The system shows the existing settings.

	The administrator updates security settings (such as password policies and user permissions). The system makes the necessary updates.
Alternative flow	If incorrect settings are input, an error message appears.
Exceptions	The system error prevents changes to security settings.
Business rules	Only administrators can change security settings.

Table 3.15: Integrate Third-Party APIs

Use case ID	UC_14
Use case name	Integrate Third-Party APIs
Actors	Admin
Description	The administrator integrates third-party APIs to enhance the system's capabilities.
Trigger	The administrator chooses the settings for API integration.
Pre-condition	The administrator possesses proper API credentials.
Postcondition	The third-party API has been successfully integrated.
Normal flow	The administrator chooses the API connection. The system requests the user to enter API keys and login details. The administrator logs in and submits the form. The system performs a verification of the credentials. The API has been integrated successfully.
Alternative flow	An error message appears if the API credentials are wrong.
Exceptions	The API service is temporarily unavailable due to downtime.
Business rules	The system must allow access solely to approved third-party APIs.

Table 3.16: View Feedback

Use case ID	UC_15
Use case name	View Feedback
Actors	Admin
Description	The administrator evaluates the comments and suggestions provided by users.
Trigger	The administrator chooses the feedback review feature.

Pre-condition	Feedback can be accessed within the system.
Postcondition	The administrator sees the feedback displayed.
Normal flow	The administrator chooses the "View Feedback' option. The system collects all feedback and presents it to users. The administrator evaluates the user.
Alternative flow	If there is no feedback, the system will alert the administrator.
Exceptions	A database error is stopping the feedback from loading.
Business rules	It's important to keep feedback securely stored and make sure to review it carefully regularly.

Table 3.17: Monitor Performance

Use case ID	UC_16
Use case name	Monitor Performance
Actors	Admin
Description	The administrator keeps an eye on the system's performance indicators.
Trigger	The administrator chooses the system monitoring feature.
Pre-condition	The system is operational.
Postcondition	Performance information is shown for review.
Normal flow	The administrator chooses the "Monitor Performance" option. The system gathers data on how well things are performing. The administrator monitors the system's performance and keeps track of the logs.
Alternative flow	If the logs can't be accessed, the system will alert the administrator.
Exceptions	The system cannot create reports because of an error.
Business rules	Only designated administrators have permission to view the performance data.

3.2. Functional Requirements

The Urdu Flow app offers features like speech recognition, translation, transcription, and summarization to improve language accessibility and communication. It is organized into two primary sections:

3.2.1. User Interface Module

The Urdu Flow app features a simple, easy-to-use design that makes navigation a breeze for everyone. It's specially optimized for smartphones, providing a smooth experience whether you're using speech recognition, translation, transcription, or summarization.

3.2.1.1. Speech and Text Processing Module

- **Convert Speech to Text:** Turn spoken words into editable text easily with speech recognition.
- **Transcribe Audio Files:** Let users upload audio files and receive accurate text transcriptions effortlessly.
- **Translate Content:** Support translation across multiple languages, especially Urdu-English and English-Urdu, so users can smoothly convert text or speech.
- **Summarize Content:** Quickly create summaries of lengthy text or spoken content, helping to make information more manageable and accessible.

3.2.1.2. File Management Module

- **Upload Files for Processing:** Allow users to upload audio or text documents for conversion, summarization, transcription, or translation.
- **Process and Convert Files:** Support for various file types ensures seamless handling, including format conversions and extracting text as needed.
- **Download and Share:** Once processed, users can download their files in multiple formats, such as PDF, Word, or plain text, for any future use.

3.2.1.3. User Profile Management

- **Updating User Profiles:** Users can modify their personal information, set their preferences, and upload files as needed.
- **Secure Authentication:** We use multi-factor authentication (MFA) to ensure that an additional layer of security protects user logins.
- **Providing Feedback:** Users are encouraged to share reviews and suggestions to help us enhance the system's performance and user experience.

3.2.2. Admin Modules

3.2.2.1. User Management Module

- **Manage User Accounts:** Admins can easily add new user accounts, update current ones, and remove any accounts when needed. They also have the ability to assign

personalized roles and permissions to each user, helping to keep the system secure and customized to their needs.

- **Monitor User Performance:** By paying close attention to how users interact with the system, we can gather helpful insights that allow us to improve our services and create a smoother, more enjoyable experience for everyone.

3.2.2.2. Security and Configuration Module

- **Secure Authentication Mechanisms:** Implement robust security policies, such as using encrypted passwords and careful session management, to help protect your system effectively.
- **Configure Security Settings:** Let admins easily set access controls, manage authentication options, and keep security strong protocols.

3.2.2.3. Analytics and Feedback Module

- **View User Feedback:** Gather and thoughtfully review feedback to enhance system features and attentively address user concerns.
- **Monitor System Usage:** Create reports on how the system is doing, including usage trends and user activities, to keep everything running smoothly. This way, everyone stays in the loop and any needed adjustments can be made easily improvement.

3.3. Non-Functional Requirements

Non-Functional Requirements outline the important qualities that the system needs to have, like being reliable, secure, and efficient, helping everything run smoothly and safely.

3.3.1. Usability

The system should be easy to use, with a clear and simple design that makes navigation and accessing features straightforward and hassle-free.

It offers accurate and easy-to-understand transcriptions and translations, providing users with a seamless experience when converting Urdu speech to English.

3.3.2. Performance

The app needs to quickly convert speech to text so users don't have to wait long, making it more user-friendly.

To optimize the system, it's important to keep an eye on key performance indicators like how quickly responses are generated, the overall processing speed, and the system's workload. Monitoring these metrics regularly helps in making informed improvements.

3.3.3. Security

Access to system features is reserved for registered users, who can enjoy secure authentication processes. This ensures everyone's safety and peace of mind.

All information, whether it is stored or sent somewhere, should be protected by encryption that follows industry best practices.

The system needs to incorporate firewalls and intrusion detection tools to help protect against cyber threats.

3.3.4. Portability

It's great that it supports exporting transcriptions and translations in various formats like TXT, PDF, and DOCX. This makes sharing your work simple and ensures compatibility across different platforms.

The app should be built in a flexible way so that it can easily be updated in the future to include more languages and features.

It's important to optimize for mobile environments so that it runs smoothly across different Android versions and devices, helping everyone have a seamless experience.

3.3.5. Reliability

The system should deliver precise outcomes, making sure that each part, from recognizing speech to managing files, functions smoothly without mistakes.

The system is crafted to provide accurate and dependable speech recognition and translation services. It aims to produce Urdu transcriptions and English translations with very few errors, ensuring clarity and trustworthiness.

It's a good idea to keep your system updated regularly so it runs smoothly and stays current with the newest technological improvements.

CHAPTER 4

DESIGN AND ARCHITECTURE

4. Design and Architecture

The overall design and architectural framework of the Admin Panel-Based AI System define how different system modules interact, how user and society data is managed, and how analytics, transcription, and summarization processes are handled. The architecture ensures scalability, modularity, and smooth coordination between the user interface, backend services, database, and AI processing modules, enabling efficient administration and control across the platform.

4.1. System Architecture

The structure of the system explains the working of the system and describes the components of the Urdu Flow, their relationships, and how they interact with each other, with this architecture.

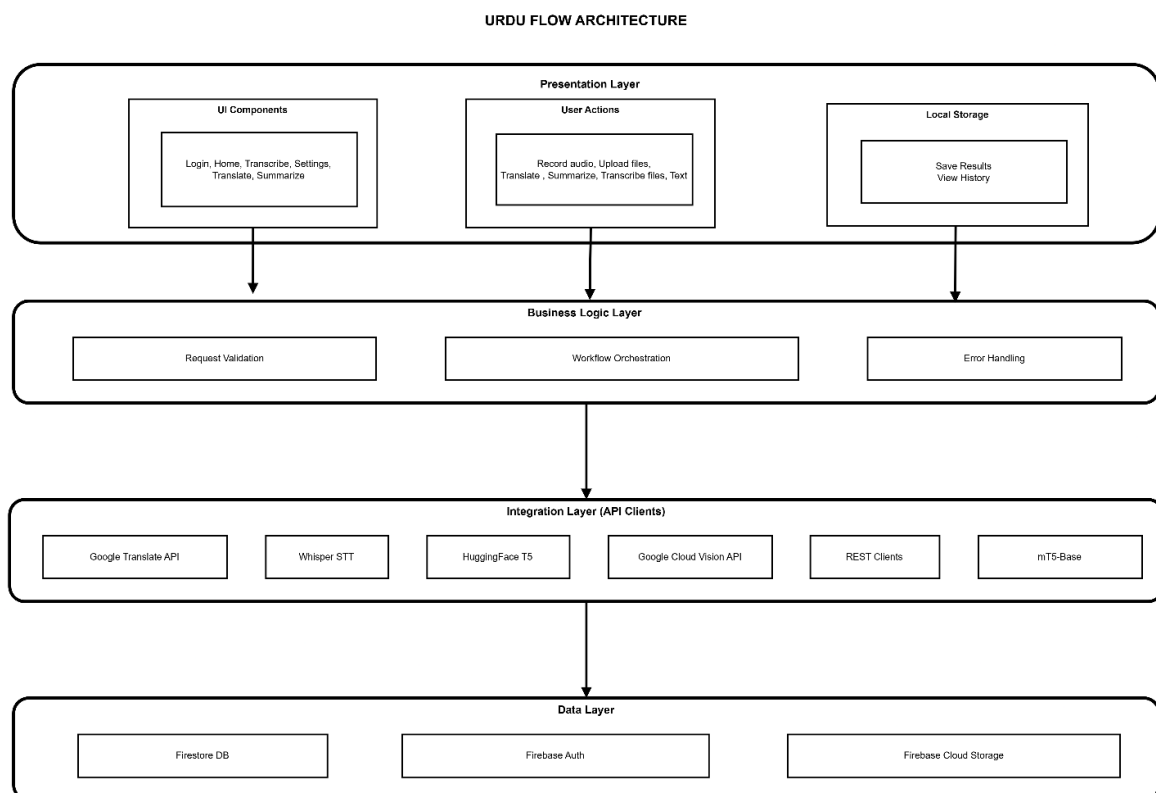


Figure 4.1: Architecture Diagram

The “Urdu Flow” system adopts a modular four-tier architecture consisting of the presentation layer, business logic layer, integration layer, and data layer. The presentation layer comprises the mobile user interface, allowing users to interact with features such as speech-to-text conversion, translation, transcription, and summarization in both Urdu and English. The

business logic layer manages application workflows, including input validation, processing logic, and result formatting for NLP tasks. The integration layer serves as the bridge to third-party services like Google Translate, Whisper STT, Tesseract OCR, and Hugging Face NLP models. The data layer is responsible for securely storing user data, transcriptions, summaries, and managing authentication via Firebase services. This architecture supports scalability, enhances system security, and simplifies future maintenance and updates.

4.2. Data Representation

In Urdu Flow, data is organized and stored using structured collections within Firebase Firestore and Firebase Storage. User profiles, sessions, uploaded files, and processed results follow predefined schemas, ensuring consistent formatting and efficient retrieval. This structured representation enables smooth handling of operations such as speech transcription, translation, summarization, and document processing. By encoding data in an organized format and decoding it during processing, the application ensures accurate output generation, reliable access to user history, and an overall seamless user experience.

4.3. Process Flow

The Process Flow Representation illustrates the step-by-step sequence of actions and interactions between different components in the Urdu Flow. It provides a visual overview of how users interact with the platform to perform various tasks.

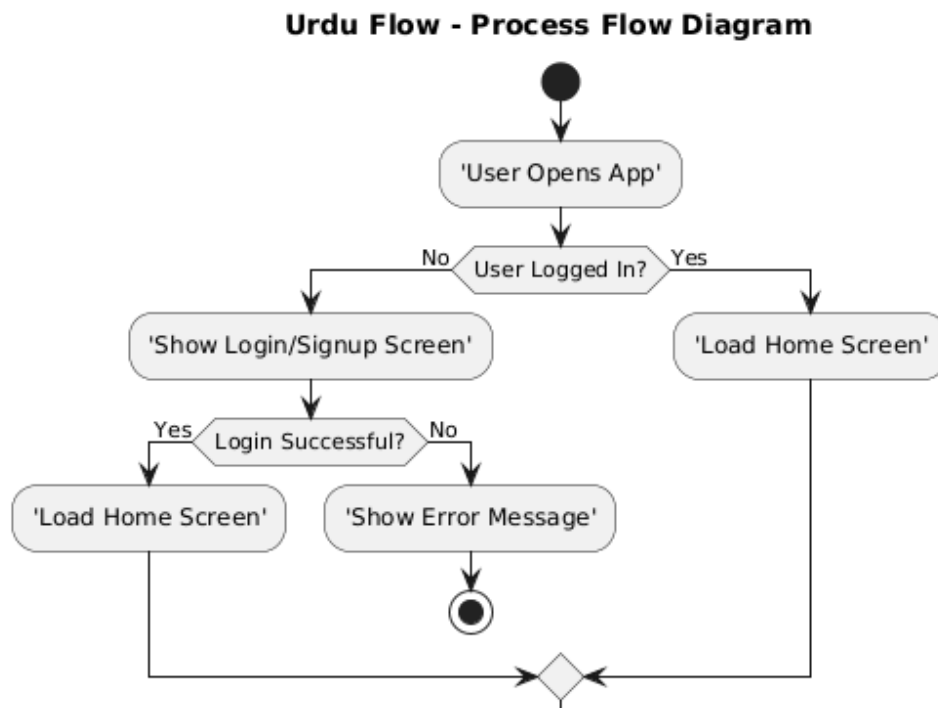


Figure 4.2: Process Flow Diagram During Authentication

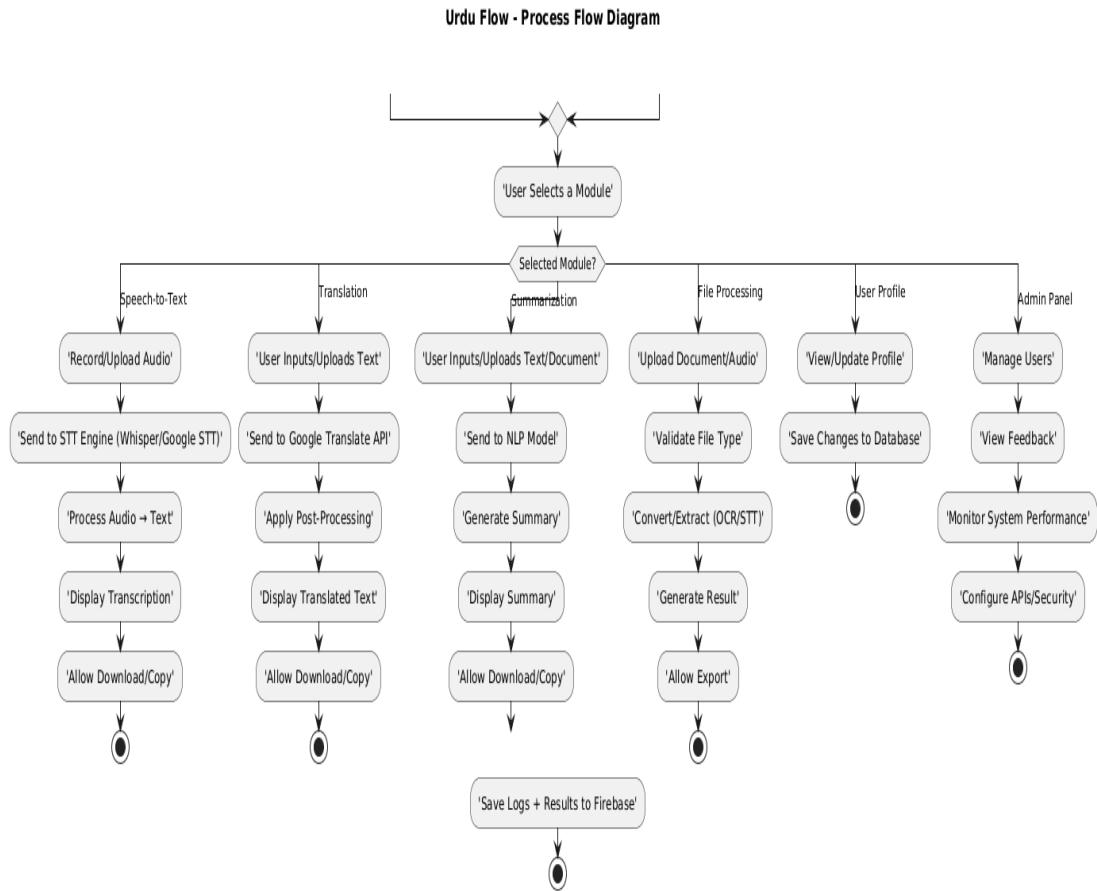


Figure 4.3: Process Flow After Authentication

4.3.1. User Registration and Login

- The user creates an account using the registration form and logs into the app.
- After successful authentication, the user is taken to the home screen where all modules are available.

4.3.2. Using the Main Modules

- **Speech-to-Text:** Users can record audio or upload an audio file to convert spoken Urdu or English into text.
- **Translate Text:** Users can type or upload text and translate it between Urdu and English.
- **Transcription:** Users can upload audio files or documents to extract text. OCR is applied for images or scanned files.

- **Summarize Content:** Users can provide long text or transcribed content to generate a concise summary.

4.3.3. Input Handling

- If a module requires input, the user either types text, records audio, or uploads a file.
- When required content is missing, the system prompts the user to provide text, audio, or documents.

4.3.4. Processing the Request

- The system triggers the appropriate processing service based on the selected module.
- Speech recognition models handle audio transcription.
- Translation uses integrated APIs with post-processing for Urdu and English.
- Summarization uses NLP models to produce short, meaningful summaries.
- Extracted or uploaded data is processed through the business logic layer.

4.3.5. Displaying the Output

- The processed results are shown on the results screen.
- Users can view, copy, download, or save the output for later use.
- Saved results can be accessed from the user's history section.

4.3.6. Admin Panel Operations

- Admins can monitor system performance, check logs, and review user activities.
- They can manage API configurations, update system settings, and review feedback submitted by users.
- Admins also oversee data storage and ensure the processing modules function properly.

4.3.7. Logging Out

- After completing their tasks, the user logs out of the application.
- Logging out ends the current session and returns the user to the login screen.

4.4. Sequence Diagram

These figure shows how the system will work through sequence diagrams, task after task sequence wise. It gives a complete description of the working of URDU FLOW.

Phase 1: Authentication Module

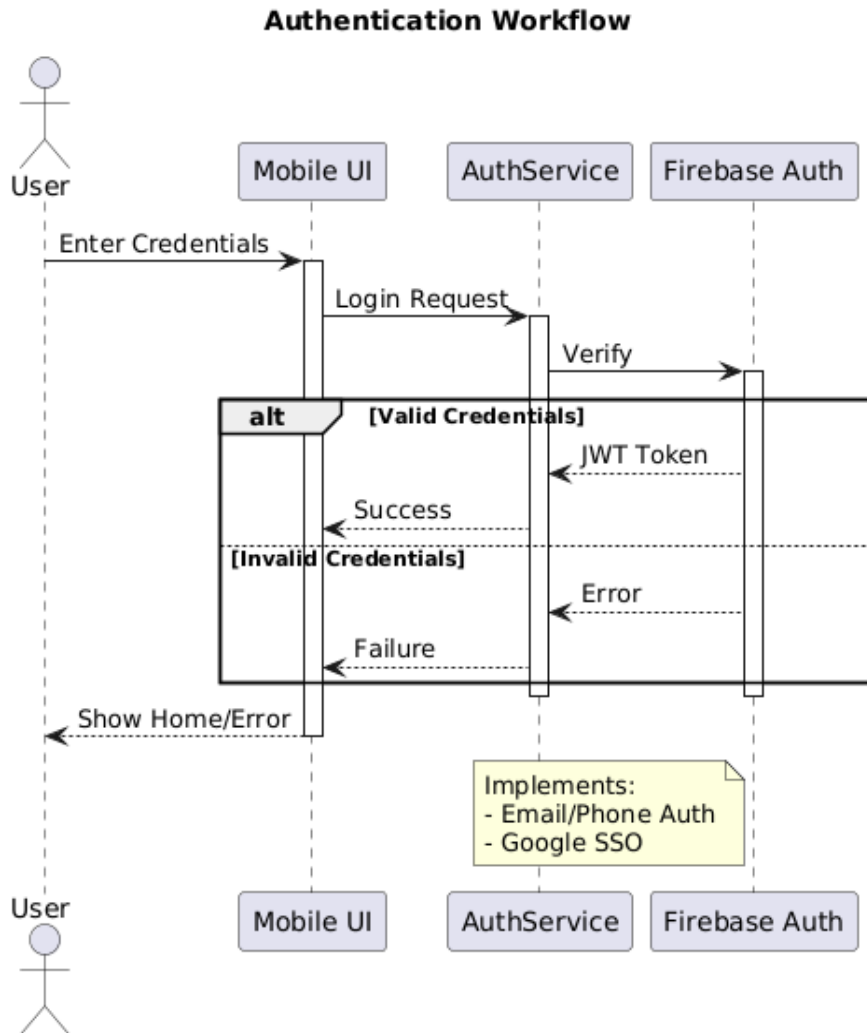


Figure 4.4: Sequence Diagram of Urdu Flow Authentication Module

The sequence diagram represents the authentication process in the Urdu Flow application. The user begins by entering their email or phone number along with their password through the mobile interface. Once the credentials are submitted, the app sends a login request to the authentication service. The authentication service then communicates with Firebase Authentication to verify the provided information.

Firebase checks whether the credentials match an existing user account. If the credentials are valid, Firebase sends back a successful response along with the authentication token. The app then grants access and redirects the user to the home screen where language processing features

such as transcription, translation, and summarization become available. If the credentials are invalid, Firebase returns an error response, and the app displays a failure message to the user.

This structured workflow ensures that only authenticated users can access the system, maintaining secure entry into the Urdu Flow platform and protecting all language processing modules and stored data.

Phase 2: Summarization Module

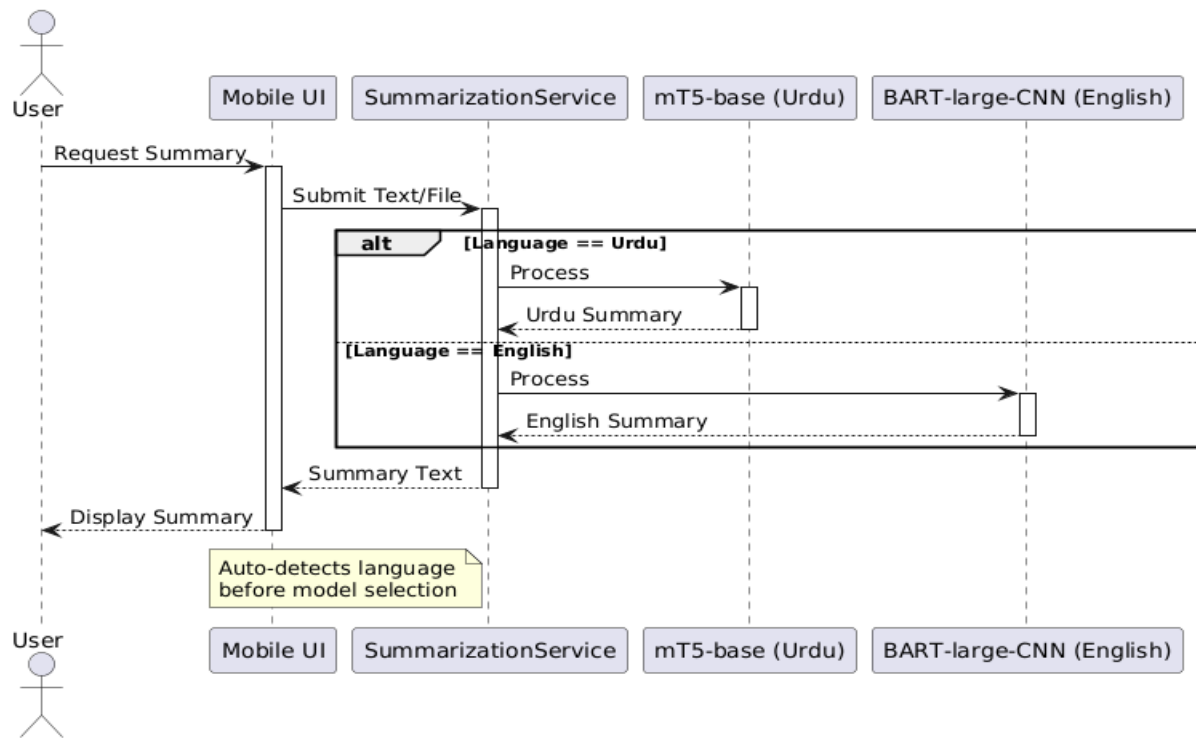


Figure 4.5: Sequence Diagram of Urdu Flow Summarization Module

This sequence diagram represents the interaction between the user, the Urdu Flow mobile application, the Summarization Service, and the NLP models used for Urdu and English summarization. The process begins when the user requests a summary. The app prompts the user to provide input, which can be either typed text or an uploaded file. Once the user submits the content, the application forwards it to the Summarization Service.

Before processing, the system automatically detects whether the input is in Urdu or English. If the text is in Urdu, the Summarization Service sends the content to the mT5-base Urdu summarization model, which processes the input and generates an Urdu summary. If the text is in English, the system sends it to the BART-large-CNN model, which produces an English summary. After the selected model finishes processing, the summarized text is returned to the

application. The app then displays the summary to the user, allowing them to read or save the output. Throughout this process, the system ensures that the correct model is chosen based on language detection, providing accurate summarization for both Urdu and English content.

Phase 3: Transcription Module

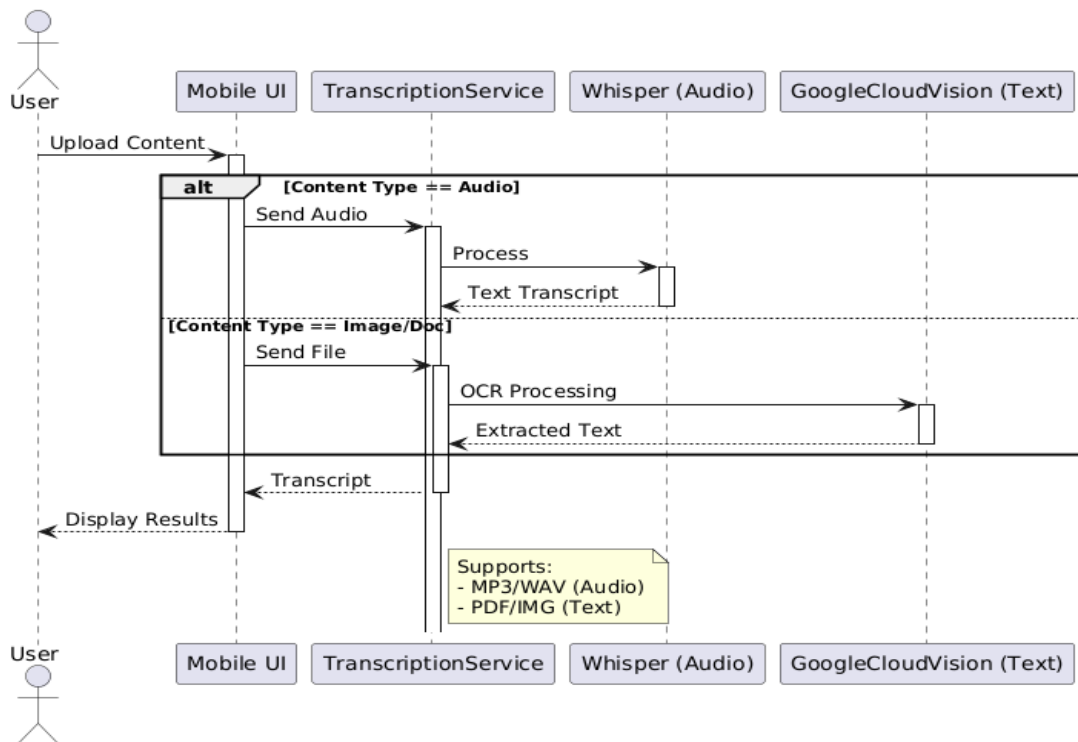


Figure 4.6: Sequence Diagram of Urdu Flow Transcription Module

This sequence diagram shows how the user interacts with the Urdu Flow app to transcribe audio or document files. The process begins when the user uploads either an audio file or an image or document. If the input is audio, the app sends it to the Whisper model, which processes the recording and returns the text transcript. If the input is a document or image, the file is sent to the OCR service, which extracts the text.

Once the transcription is completed, the result is sent back to the app and displayed to the user. The user can then view or save the extracted text. This flow ensures that Urdu Flow can accurately handle both audio transcription and document text extraction.

Phase 4: Translation Module

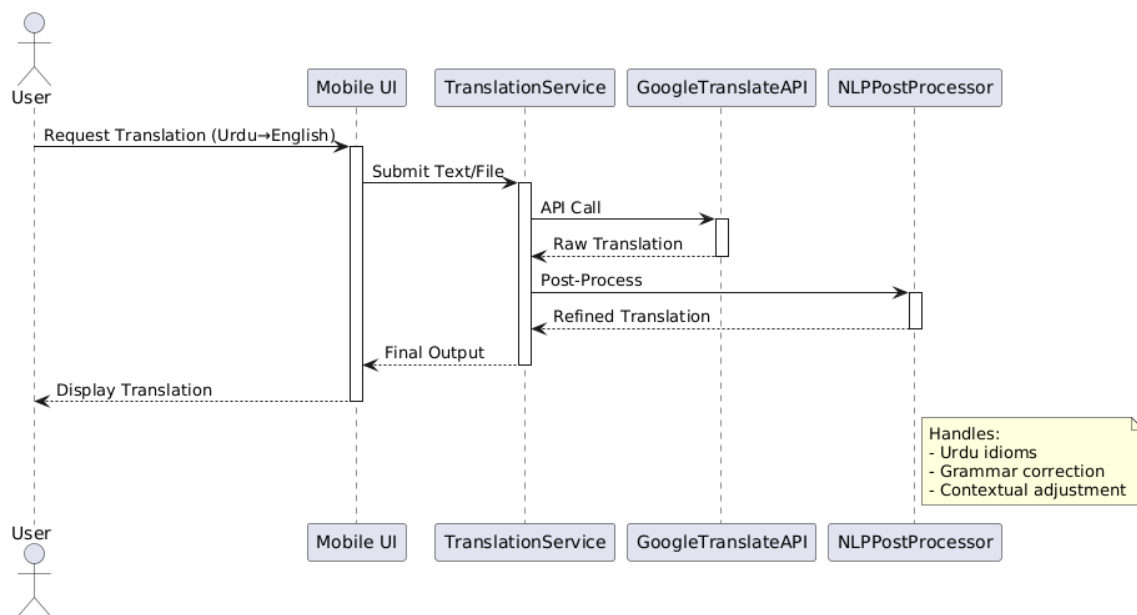


Figure 4.7: Sequence Diagram of Urdu Flow Translation Module

This sequence diagram shows how Urdu Flow processes a translation request. The user submits text or uploads a file through the mobile interface, and the content is sent to the Translation Service. The service forwards the input to the Google Translate API, which returns the raw translated text. This text is then refined by the NLP Post-Processor to correct grammar, adjust context, and handle Urdu-specific expressions. Once processing is complete, the final translation is returned to the app and displayed to the user. This ensures accurate and polished translation results for both Urdu and English content.

Phase 5: File Management Module

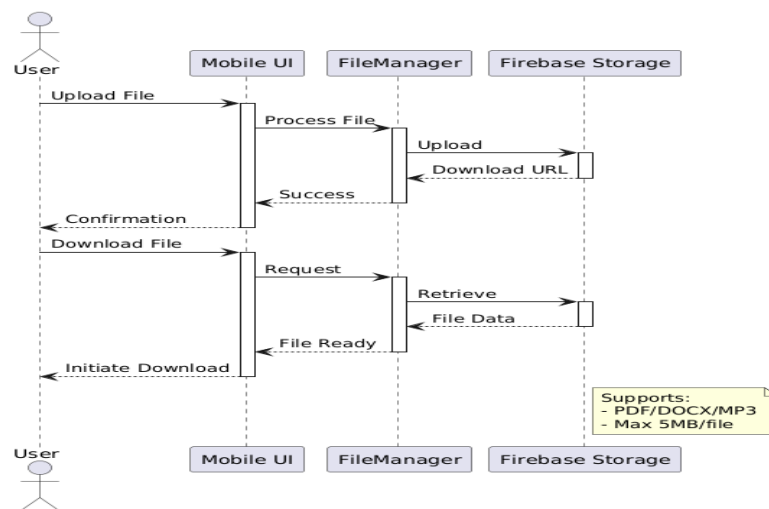


Figure 4.8: Sequence Diagram of Urdu Flow File Management Module

This sequence diagram shows how Urdu Flow manages file uploads and downloads. The user uploads a file through the app, which is processed by the File Manager and then stored in Firebase Storage. Once the upload is complete, the app receives a download link and confirms the upload to the user.

When the user wants to download a file, the File Manager requests the file from Firebase Storage. After the file is retrieved, the app notifies the user, who can then download it to their device. This process provides reliable handling of supported file formats within the application.

Phase 6: Admin Operations

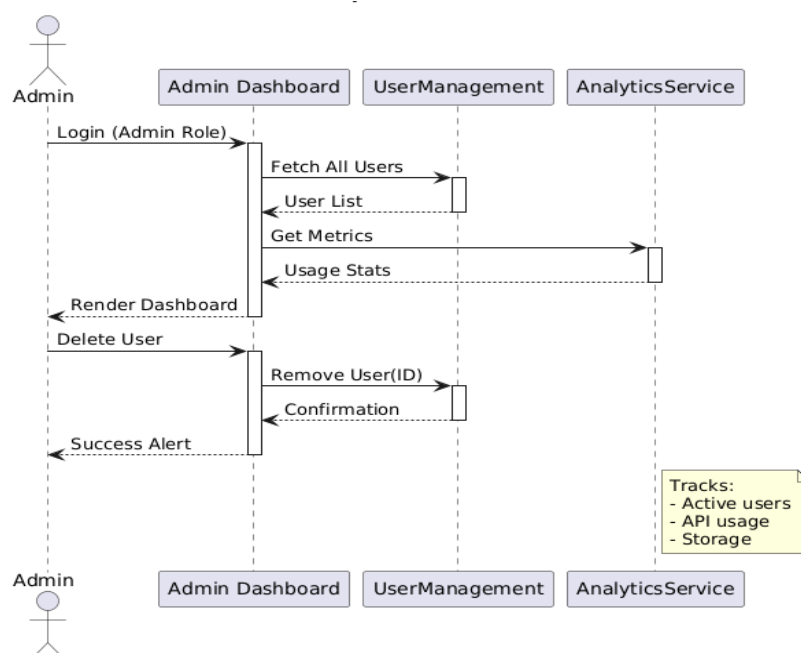


Figure 4.9: Sequence Diagram of Admin Operations

This sequence diagram shows how the admin manages the Urdu Flow system. After logging in, the admin dashboard loads by retrieving all users and system metrics such as usage statistics and API activity. The admin can review this data and perform actions like removing a user. When a deletion request is made, the User Management service processes it and sends back a confirmation. The app then notifies the admin that the operation was successful. This process ensures efficient monitoring and control of the system by administrators.

4.5. Class Diagram

This class diagram is a graphical representation of the system's classes, their attributes, and the relationships between them. It elaborates structure of the Urdu Flow and shows how the different classes in the system are related to each other.

4.5.1. Class Diagram for Urdu Flow

This class diagram shows how an Urdu processing system works from the moment a user uploads a file until the system generates the final transcription, translation, or summary. The system starts with a general Account class, which contains basic information like email, password, and when the account was created. Both regular users and admins are built on top of this account structure. Regular Users have their own preferences and activity details, and they can request transcriptions and later download the processed results. Admins, on the other hand, have higher privileges. They can manage the system, suspend users if need, and check system performance.

Users interact with the system by uploading different types of files audio, text, or images. These files are represented by the MediaFile class, which stores information such as the file type, size, and language (Urdu or English). Before processing, the file can be validated or compressed to ensure it meets system requirements. Once uploaded, these files move into the processing pipeline.

The system processes files through services that implement a shared interface called IProcessor. This interface guarantees that every processing service can accept a media file and return a result. There are different processors for different tasks. The SpeechToTextService converts speech into written text, often using external tools like Whisper to handle accents and improve accuracy. The TranslationService handles translating text between languages and may rely on tools like Google Translate for language detection. Meanwhile, the SummarizationService

focuses on reducing long text into shorter, meaningful summaries using Urdu and English models.

After processing is completed, the system generates a result based on the type of processing that was performed. These results all inherit from a common ProcessingResult class, which stores helpful information such as how long the processing took. The specific result types include TranscriptionResult (for speech-to-text outputs), TranslationResult (for translated content), and SummaryResult (for summarized text). Each of these holds the relevant data for example, original and target text for translations or before-and-after text lengths for summaries.

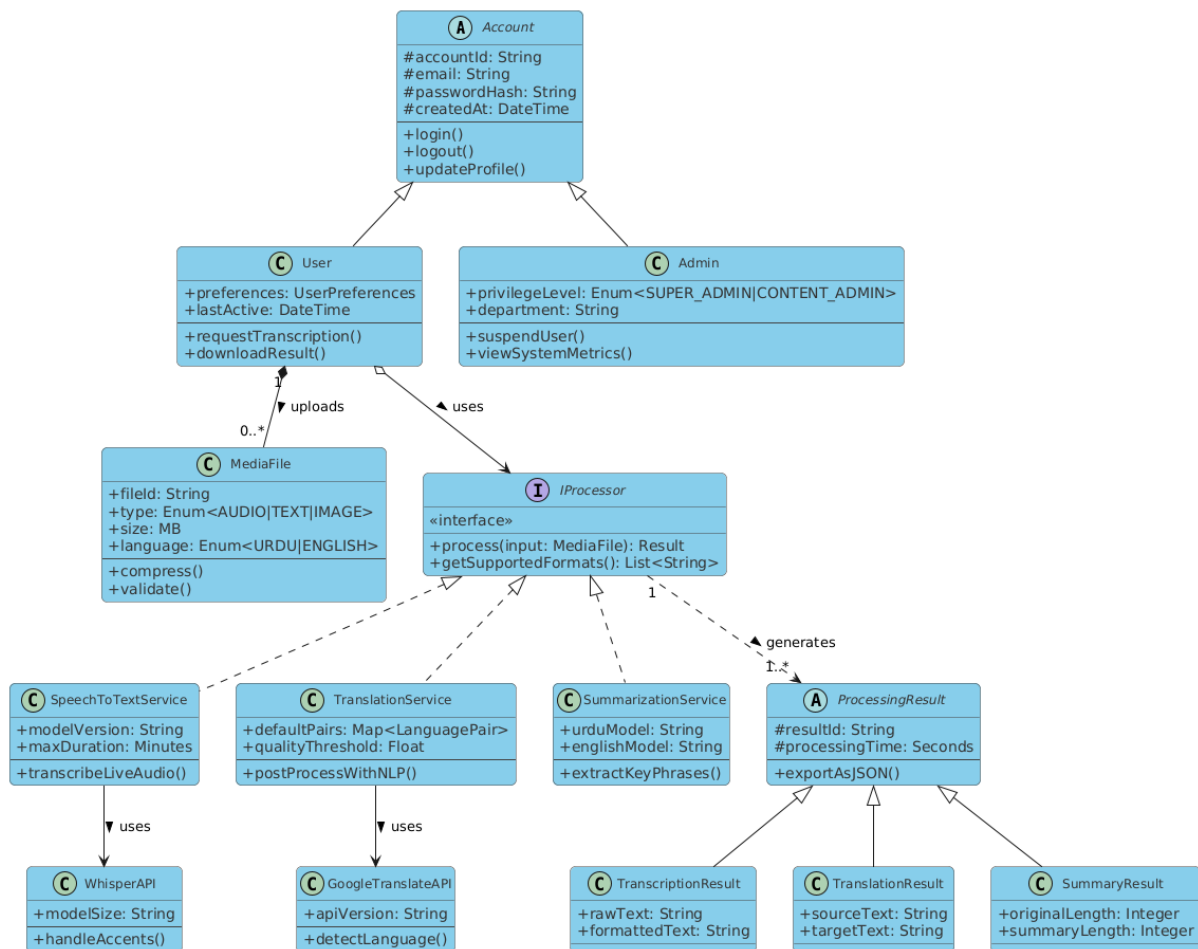


Figure 4.10: Class Diagram

4.6. Summary

The class diagram outlines the structural design of the Urdu Flow application. Users and admins interact with the system through authenticated accounts. Uploaded files are handled as Media File objects, which are then processed by specialized modules for transcription, translation, and summarization. All processing modules follow a shared interface to ensure consistency and scalability. The results of these operations are stored in structured output

classes for easy retrieval. This design supports modular, maintainable, and extensible development for Urdu Flow’s language-processing features.

CHAPTER 5

IMPLEMENTATION

5. Implementation

The implementation of Urdu Flow utilizes efficient algorithms and third-party machine learning models to ensure accurate processing of language-related tasks including transcription, translation, and summarization. Below are key algorithms used in different modules of the system:

5.1. Algorithm

5.1.1. User Authentication

Function: authenticate User (username, password)

Input: username (string), password (string)

Output: user (object) or error (string)

Pseudocode:

1. BEGIN
2. Retrieve user record from the database based on the provided username.
3. If a user with the given username exists:
 - a. Compare the provided password with the stored password hash.
 - b. If the passwords match, return the user object.
 - c. If the passwords do not match, return an error message indicating "Incorrect password".
4. If no user is found, return an error message indicating "User not found".
5. END

5.1.2. Speech-to-Text (Transcription) Algorithm

Function: TranscribeAudio(audioFile)

Input: AudioFile (binary audio file or recorded stream)

Output: Transcript (string) or confidence_score (float)

Pseudocode:

1. BEGIN
2. Validate that the audio file is in a supported format.
3. Send audioFile to the Whisper STT Model.
4. Receive:
 - a. transcribed_text
 - b. confidence_score

- c. language_detected
5. Format the text for readability.
6. Return the final transcript and confidence score.
7. END

5.1.3. Translation Algorithm

Function: TranslateText(inputText, targetLanguage)

Input: InputText (string), targetLanguage (string: "ur" or "en")

Output: TranslatedText (string)

Pseudocode:

1. Validate targetLanguage.
2. Prepare HTTPS request to Google Translate API.
3. Send inputText for translation.
4. Receive translated_text from API.
5. Apply optional post-processing using:
 - regex cleanup
 - domain-specific corrections
 - LLM refinement
6. Return cleaned translatedText

5.1.4. Summarization Algorithm

Function: Summarize Content(content, summaryLength)

Input:

- Content (string)
- SummaryLength (optional integer)

Output: Summary (string)

Pseudocode:

1. BEGIN
2. Tokenize content.
3. Pass tokens to the Transformer Summarization Model (T5/mT5).
4. Generate abstractive summary based on summaryLength or default settings.
5. Retrieve:

- a. summary_text
 - b. optionally: key sentences, keywords
6. Return formatted summary_text.
7. END

5.1.5. Chat Interaction

Function: uploadFile(user, file)

Input:

- user (object)
- file (object)

Output: File_url (string) OR error (string)

Pseudocode:

1. Validate that user is authenticated.
2. Validate file type and size.
3. Upload file to Firebase Storage.
4. Retrieve generated file_url.
5. Save metadata in database.
6. Return file_url.

5.1.6. Process & Convert Files Algorithm

Function: processFile(file, outputFormat)

Input:

- File (object)
- Output Format (string)

Output: ConvertedFile (file object)

Pseudocode:

1. Check if file format is supported.
2. Determine file type (audio, text, document).
3. If audio → transcription module

4. If document → OCR + text extraction
5. Convert extracted content into requested outputFormat (PDF/TXT/DOCX).
6. Save converted file.
7. Return convertedFile

Table 5.1: External API's

API Name	Description	Purpose	Function/Class Used
Firebase Authentication	Allows secure and easy user authentication using email/password or other methods.	Handles login, sign-up, and authentication for all users (student, admin, head).	Login Activity, Register Activity, Auth Service
Firebase Firestore	A cloud-based NoSQL database that stores and syncs app data in real time.	Stores data like events, societies, feeds, and user profiles.	Event Repository, Society Service, Feed Adapter

5.2. User Interface

The user interface (UI) of URDU FLOW is designed to be simple, intuitive, and visually appealing. Below are the main screens:

- **Figure 5.1:** The Login screen where users enter their email and password to sign in to the Urdu Flow app.
- **Figure 5.2:** The signup screen for new users to create an account.
- **Figure 5.3:** The Home screen where users can view their profile and access the main services of Urdu Flow such as Transcribe, Translate, and Summarize.
- **Figure 5.4:** The Edit Profile screen where users can update their personal details, contact information, and bio.
- **Figure 5.5:** The Summarization screen allows users to enter or upload text, select a language, and generate a concise summary using Urdu Flow's summarization feature.

- **Figure 5.6:** The Translation screen enables users to enter or upload text, choose a language, and generate translations between English and Urdu using Urdu Flow’s translation feature.
- **Figure 5.7:** The Transcription screen allows users to record or upload audio, select a language, and view the converted text produced by Urdu Flow’s speech-to-text feature.
- **Figure 5.8:** The Settings screen allows users to manage notifications, adjust privacy preferences, view app information, access help and support, and securely log out of Urdu Flow.

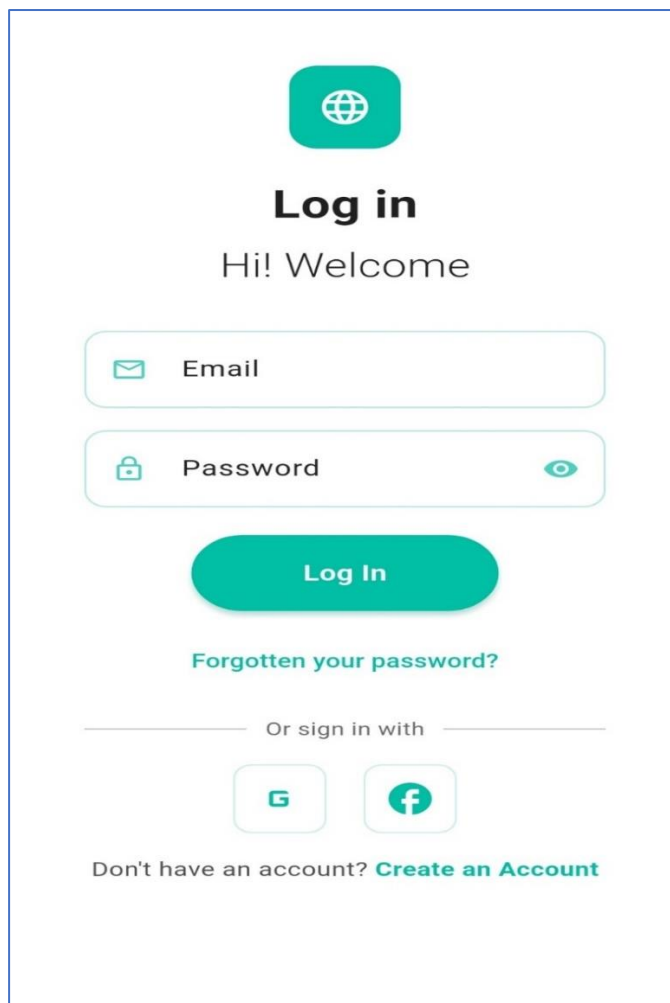
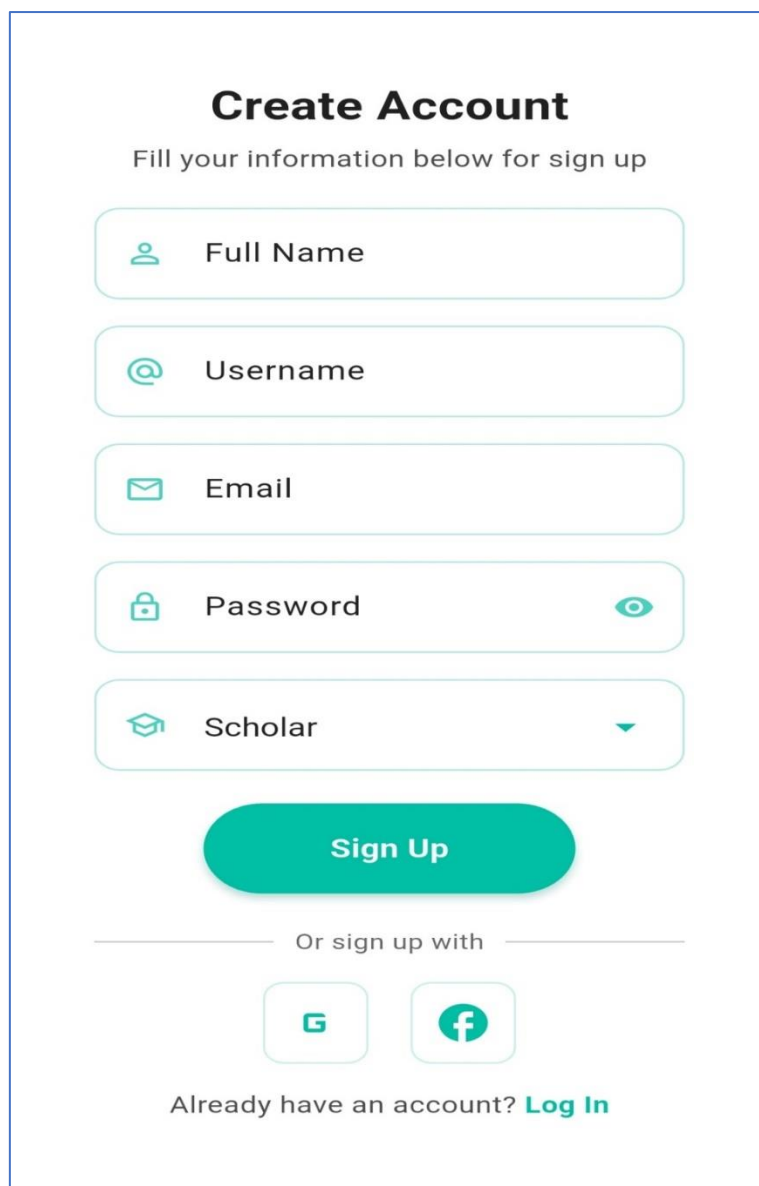


Figure 5.1: UI for Login page of Urdu Flow

The Login screen serves as the main access point for users entering the Urdu Flow application. The interface is simple, clean, and designed to help users sign in without any confusion. At the top, a bold “Log in” heading clearly informs the user of the screen’s purpose, followed by a friendly welcome message to create a comfortable first impression.

Users are provided with two input fields: one for entering their email and one for their password. Both fields are neatly styled with icons to indicate their purpose, making the form easy to understand at a glance. A “Forgotten your password?” link is placed conveniently below the password field, offering a quick way for users to recover their accounts if needed.

The green “Log In” button stands out at the center of the screen, prompting users to proceed with signing in. At the bottom of the screen, a short message is displayed for users who do not yet have an account, along with a “Create an Account” link that directs them to the registration page.



The image shows a 'Create Account' form with a light gray background and rounded rectangular input fields. The title 'Create Account' is in bold black text, followed by the instruction 'Fill your information below for sign up'. The form contains five input fields: 'Full Name' with a person icon, 'Username' with an '@' icon, 'Email' with an envelope icon, 'Password' with a lock icon and a toggle eye icon, and 'Scholar' with a graduation cap icon and a dropdown arrow. Below these fields is a large green 'Sign Up' button. Underneath the button is a horizontal line with the text 'Or sign up with'. Below this line are two social media login buttons: one with a Google 'G' logo and another with a Facebook 'f' logo. At the bottom, the text 'Already have an account?' is followed by a green 'Log In' link.

Create Account

Fill your information below for sign up

 Full Name

 Username

 Email

 Password 

 Scholar 

Sign Up

Or sign up with

Already have an account? [Log In](#)

Figure 5.2: UI for Sign Up page of Urdu Flow

The Sign-Up screen provides a simple and clean interface for new users to create an account on the Urdu Flow app. At the top, the “Create Account” heading clearly states the purpose of the page. Below it, users can enter their full name, username, email, password, and choose their role from the dropdown menu. The form fields are neatly spaced and easy to follow, making the registration process straightforward. A prominent “Sign Up” button allows users to complete their registration, while a link at the bottom guides existing users to the login page.

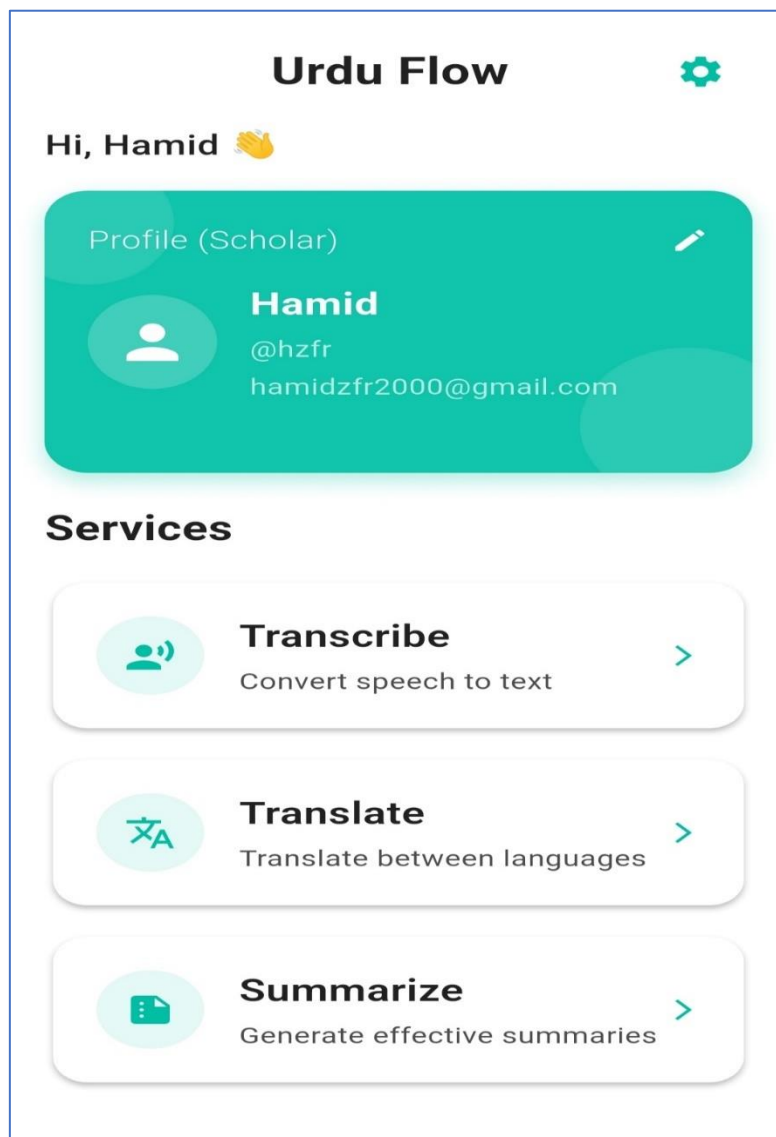
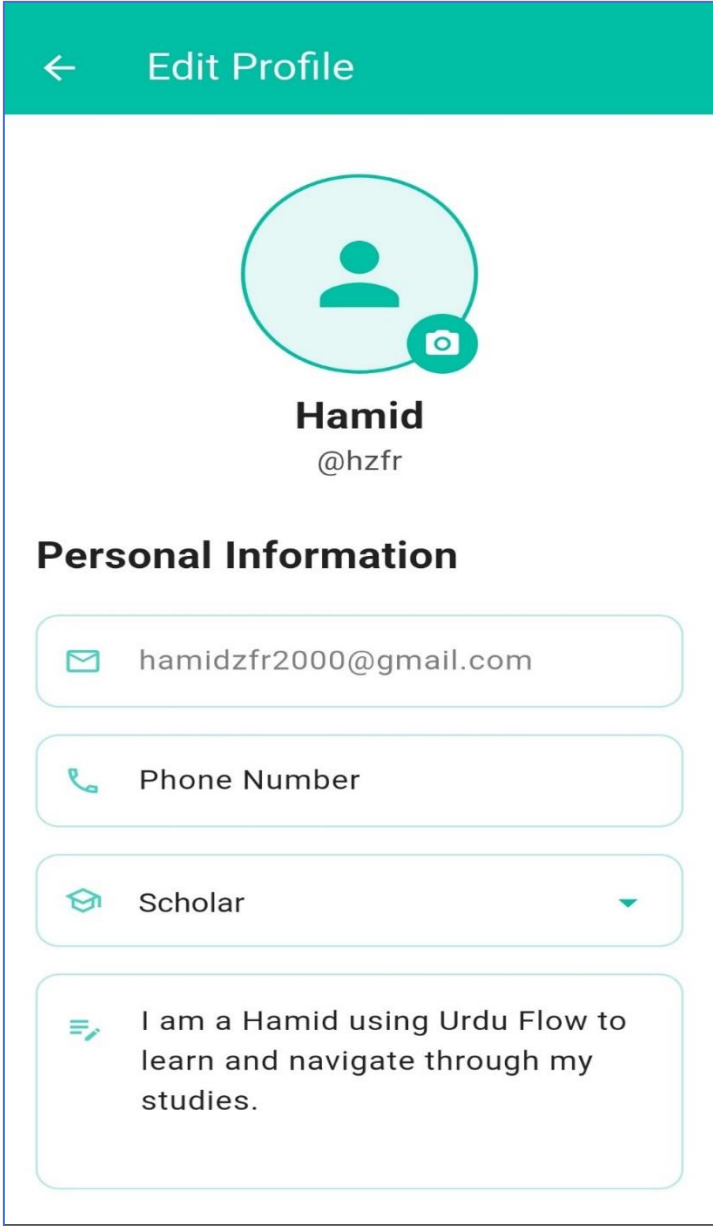


Figure 5.3: UI for Home page of Urdu Flow

The Home screen acts as the main dashboard for users in the Urdu Flow app. At the top, the app name “Urdu Flow” is displayed along with a settings icon, giving users quick access to account configurations. Just below, a personalized greeting welcomes the user by name,

creating a friendly and engaging start. The user's profile card appears prominently, showing their name, username, email, and role, with an edit option for updating profile details.

Under the profile section, the screen lists the core services offered by the app. Each service appears in a separate card-style layout with a clear label and icon. Users can choose to transcribe audio, translate text, or generate summaries through these modules. The overall layout is clean, organized, and easy to navigate, helping users quickly access the language processing tools they need.



The image shows a mobile application interface for editing a user profile. At the top, there is a teal header bar with a white back arrow and the text "Edit Profile". Below the header, the profile section features a circular profile picture placeholder with a teal person icon and a camera icon in the bottom right corner. Underneath the profile picture, the name "Hamid" is displayed in bold, followed by the username "@hzfr". Below this, the section "Personal Information" is titled in bold. It contains four input fields, each with a teal icon on the left: an email field with an envelope icon containing "hamidzfr2000@gmail.com", a phone number field with a telephone icon containing "Phone Number", a role field with a graduation cap icon containing "Scholar" and a dropdown arrow, and a bio field with a pencil icon containing the text "I am a Hamid using Urdu Flow to learn and navigate through my studies."

Figure 5.4: UI for Edit Profile page of Urdu Flow

The Edit Profile screen allows users to update their personal information within the Urdu Flow app. At the top, a simple header displays “Edit Profile,” making the purpose of the screen clear. Below it, the user’s profile picture is shown in a circular frame along with an icon indicating that it can be changed. The user’s name and username are displayed right under the profile image, giving a quick overview of their identity.

The main section of the screen contains editable fields under the heading “Personal Information.” These fields include the user’s email, phone number, and selected role, all presented in clean, clearly labeled input boxes. At the bottom, a text box allows the user to write or update their personal bio. The layout is minimal and user-friendly, ensuring that users can easily make changes to their profile details.

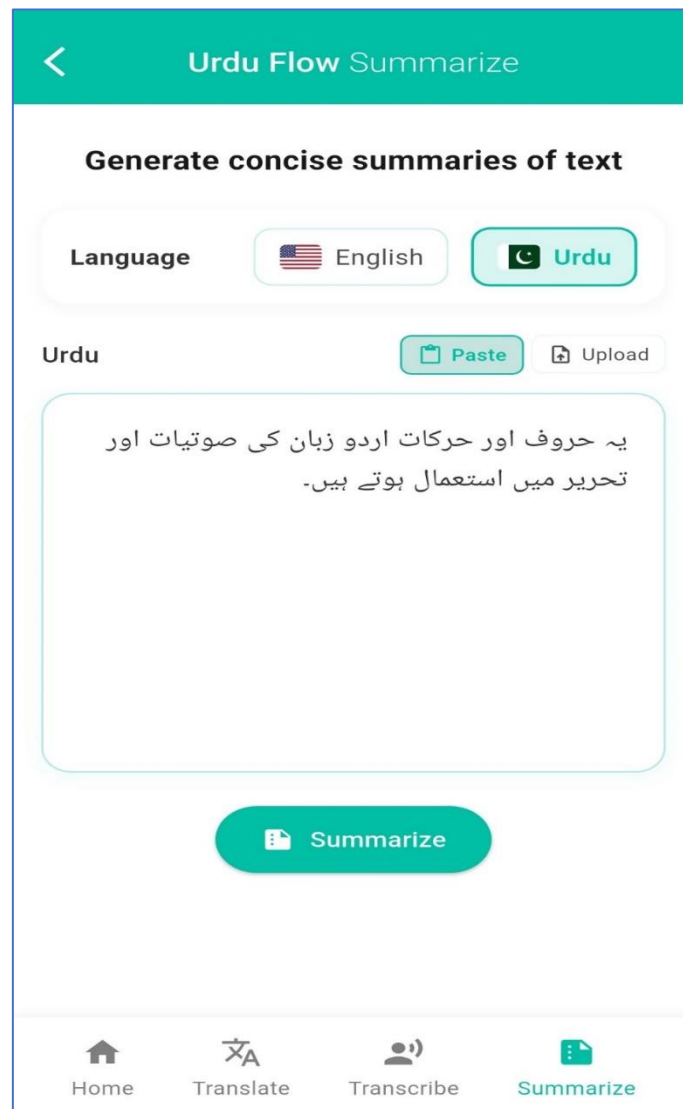


Figure 5.5: UI for Summarization page of Urdu Flow

The Summarization screen is designed to help users generate concise summaries from Urdu or English text in a simple and intuitive way. At the top, the header displays “Urdu Flow Summarize,” clearly indicating the function of the page and providing a clean navigation structure. Just below, users can choose the language they want to summarize in by selecting either English or Urdu. The chosen language is highlighted, making it easy for users to confirm their selection.

A large text input area is placed at the center of the screen, where users can directly type, paste, or upload content using the available options above the text box. These options make the screen flexible for different types of input and allow users to quickly bring in content for summarization.

At the bottom, a prominent Summarize button invites the user to process their text. Once tapped, the system will generate a shorter, meaningful summary based on the provided input. The bottom navigation bar includes shortcuts to Home, Translate, Transcribe, and Summarize, helping users switch between different features easily.

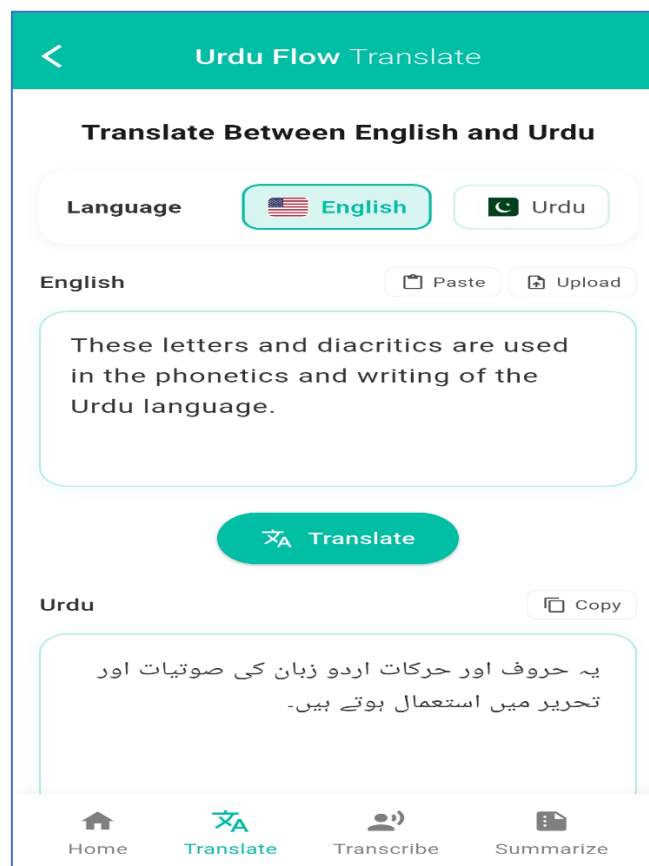


Figure 5.6: UI for Translation page of Urdu Flow

The Translation screen provides a straightforward and well-organized interface for users who want to translate content between English and Urdu. At the top, the header “Urdu Flow Translate” clearly indicates the purpose of the page and maintains a consistent layout with the rest of the app. Just below, the user is presented with language selection options for English and Urdu, with the chosen language highlighted for clarity.

The main part of the screen contains a large text box where users can type, paste, or upload the content they want to translate. This section is accompanied by clear labels, making it easy to understand which language the input text belongs to. A prominent Translate button is placed beneath the input area, guiding users to initiate the translation process.

Once the translation is complete, the output is displayed in a separate text box below, labeled according to the target language. A convenient copy option allows users to quickly save or use the translated text as needed. At the bottom, the navigation bar includes quick links to Home, Transcribe, and Summarize, offering smooth movement between different features of the app.

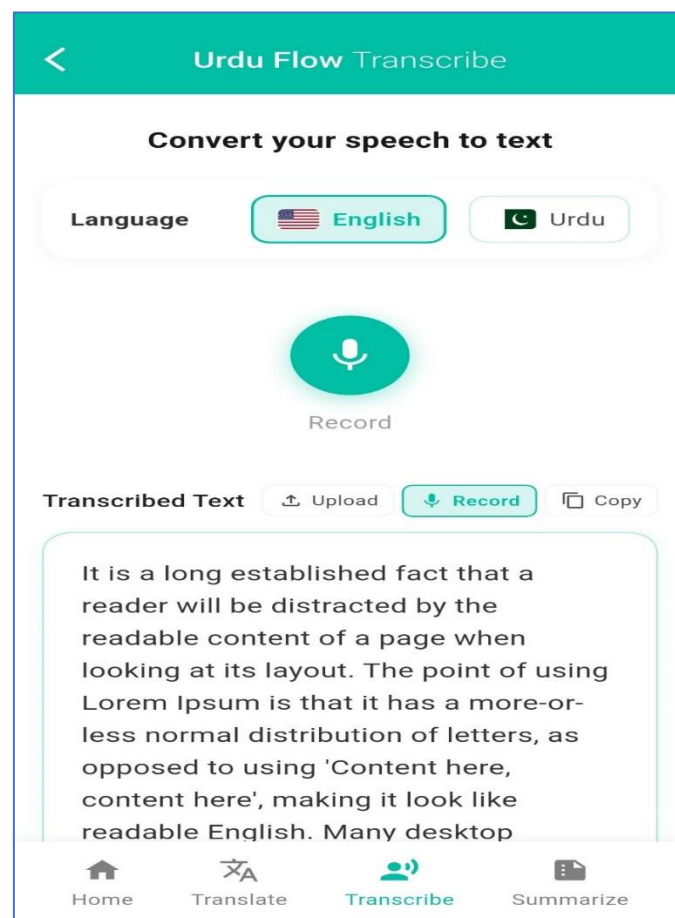


Figure 5.7: UI for Convert to Speech Page of Urdu Flow

The Transcription screen is designed to help users convert spoken audio into text in a simple and intuitive way. At the top, the header “Urdu Flow Transcribe” clearly identifies the purpose of the page. Just below, users can choose their preferred language, with options for English and Urdu displayed in a clear toggle format.

The central focus of the screen is a large microphone button that allows users to begin recording their speech. The layout is minimal and clean, making the recording action easy to understand. Once the recording is complete, the transcribed text appears in a dedicated text box below. This section includes options to upload an audio file, record again, or copy the generated text, giving users flexibility in how they interact with the feature.

At the bottom, a navigation bar provides easy access to other key modules, including Home, Translate, and Summarize. The overall layout is user-friendly, well-spaced, and designed to guide users smoothly through the transcription process.

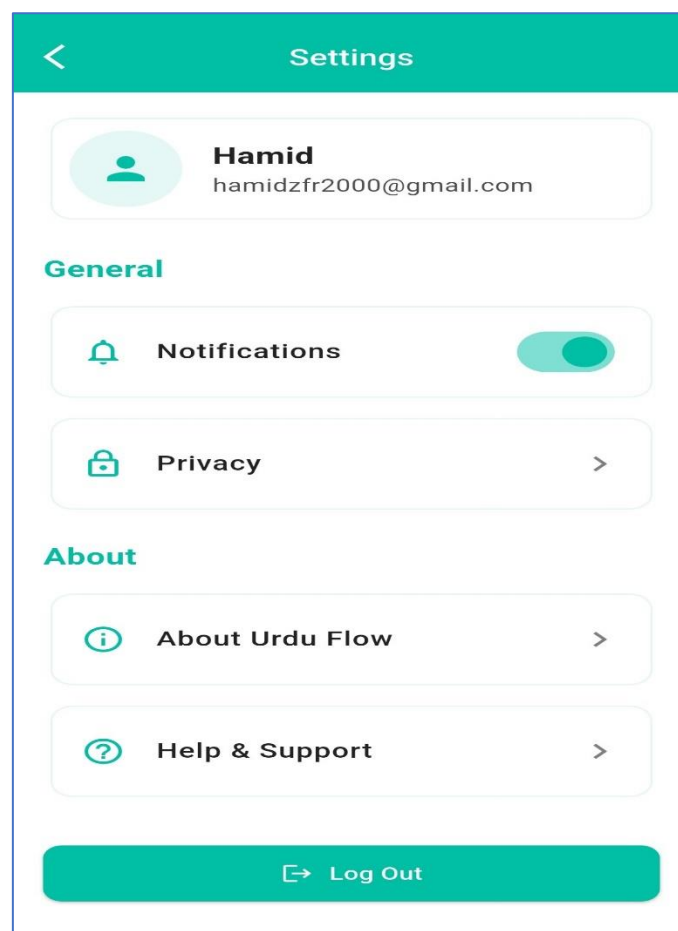


Figure 5.8: UI for Settings Page of Urdu Flow

The Settings screen of the Urdu Flow application provides users with a clear and organized place to manage their account preferences. The top of the screen features a simple header labeled “Settings,” making it easy to recognize the purpose of the page. Just below, the user’s profile information is displayed in a small card that includes their name and email, offering a quick overview of their identity within the app.

The screen is divided into sections for easier navigation. Under the General section, users can control their app notifications through a toggle switch, allowing them to enable or disable alerts as needed. There is also a dedicated Privacy option, which leads to settings related to account security and data preferences.

In the About section, users can access information about the Urdu Flow application as well as a Help and Support option for assistance or guidance. At the bottom of the screen, a clearly visible Log Out button allows users to safely exit their account whenever needed. The overall layout is simple, intuitive, and ensures a smooth experience while managing personal settings and app-related preferences.

CHAPTER 6

TESTING AND EVALUATION

6. Testing and Evaluation

Testing ensures whether the required functionality is developed and meets user needs. It is performed at different phases, with errors identified and resolved before proceeding. Testing evaluates the program's ability to meet desired results. In many software engineering methodologies, it is a separate phase after implementation, allowing fresh perspectives to identify errors more effectively.

6.1. Manual Testing

Manual Testing is a type of Software Testing where testers manually execute test cases without automation tools. It is the most basic testing method, used to find software bugs. Testers act as end users, exploring the application's features to ensure correct behavior, though they may have limited insight into the product's inner workings, potentially missing deeper issues or root causes.

6.2. System Testing

System testing ensures the application meets its requirements and quality standards. It is performed after development to verify the system works as intended and uncovers hidden errors. Types of testing include unit, functional, and integration testing, all of which must be completed before deployment.

6.3. Unit Testing

Unit testing in the Urdu Flow application focuses on evaluating each core module independently to ensure that every feature behaves correctly before integrating it into the full system. These tests cover critical components such as secure login authentication, speech-to-text transcription, Urdu-English and English-Urdu translation, document summarization, and file upload/processing operations. For example, unit tests validate user inputs like email formats, password length, audio file types, and text content to ensure they meet the expected data types and validation rules. Additionally, tests verify that UI-level actions such as tapping the "Translate," "Summarize," or "Upload" buttons correctly trigger their associated functions in the app and backend logic.

Unit Testing 1: Login as a Admin

Objective: To ensure the login form and signup form are working correctly.

Table 6.1: Login as an Admin

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-001	Admin Login with valid credentials	Username: hzfr921@gmail.com Password: Admin@123	Admin successfully logs into the system	Pass
UT-002	Admin Login with invalid credentials	Username: wrongadmin@gmail.com Password: wrong pass	Error message: "Invalid email or password"	Pass
UT-003	Admin Login with empty fields	Username: Password:	Prompt: "Please enter both email and password"	Pass

Unit Testing 2: Login as an existing user

Testing Objective: To ensure the login and signup forms are functioning correctly for existing users in the Urdu Flow app.

Table 6.2: Login as an existing user

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-001	An existing user login with valid credentials	Email: Abdulmanan2806@gmail.com Password: mani123 Email: shanzanwr@gmail.com Password: shanza22	Proceeds to services page	Pass

UT-002	An existing user login with invalid credentials	Email: ward@gmail.com Password: wrong pass	Error message: "Invalid email or password"	Pass
--------	---	---	--	------

Unit Testing 3: Logout as an Admin

Testing Objective: To ensure that the Parent successfully log out from the system.

Table 6.3: Logout as an Admin

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-001	Click the logout button and logout	Logout	Clears admin in-data from memory and proceeds to login page	Pass

Unit Testing 4: Logout as a User

Testing Objective: To ensure that the society president successfully logged out from the system.

Table 6.4: Logout as User

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-001	Click the logout button and logout	Logout	Clears user in-app data from memory and proceeds to login page	Pass

6.4. Functional Testing

The functional testing will take place after the unit testing. In this functional testing, the functionality of each of the modules is tested. This is to ensure that the system produced meets the specifications and requirements.

Table 6.5: Login as Admin

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
FT-001	Admin login with valid credentials	Email: hzfr921@gmail.com Password: Admin@123	Admin Successfully logged in to the system	Pass
FT-002	Admin Login with invalid credentials	Username: wrongadmin@gmail.com Password: wrong pass	Error message: "Invalid credentials"	Pass
FT-002	Translate text (English)	Text: I am feeling happy today.	میں آج خوش محسوس کر رہا ہوں	Pass
FT-003	Translate (Urdu)	Text: حال ہی میں آپ کیسی ہو؟	How are you doing lately?	Pass
FT-004	Transcription	Audio file: counting.mp3	1 2 3 4 5 6	Pass
FT-005	Summarization (Urdu or English)	Image Document of 14 lines	Five to Seven Lines of summary	Pass
FT-006	Click the logout button and logout	Logout	Admin and users Successfully exit from the system	Pass

The functional testing table outlines a comprehensive set of test cases conducted to verify the core features of the Urdu Flow application. Test Case **FT-001** validates the admin login process using correct credentials (hzfr921@gmail.com, Admin@123), confirming that the administrator can successfully access the system without any authentication issues. Test Case **FT-002** evaluates the system's response to incorrect login attempts by entering an invalid email

and password. As expected, the system displays the appropriate error message “Invalid credentials,” ensuring secure access control. A second entry labeled **FT-002** (which should ideally be **FT-003**) assesses the English-to-Urdu translation functionality. The input text “I am feeling happy today.” is processed accurately, and the system returns the correct Urdu translation “میں آج خوش محسوس کر رہا ہوں,” demonstrating that the translation module is working effectively. Test Case **FT-003** verifies the Urdu-to-English translation feature by inputting the Urdu sentence “حال ہی میں آپ کیسی ہو؟”, the system successfully translates it to “How are you doing lately?”, confirming proper bidirectional translation capability. Test Case **FT-004** examines the transcription module using an audio file (counting.mp3). The system correctly identifies and transcribes the spoken content as “1 2 3 4 5 6,” validating the accuracy of the speech-to-text component. Test Case **FT-005** evaluates the summarization module by uploading an image document containing fourteen lines of text. The system processes the document and generates a concise summary consisting of five to seven lines, meeting the expected summarization criteria. Finally, Test Case **FT-006** verifies the logout functionality. Clicking the logout button successfully ends the session for both admin and normal users, redirecting them back to the login screen. All test cases passed successfully, demonstrating that Urdu Flow’s authentication, translation, transcription, summarization, and logout features are functioning correctly and aligning with the system’s intended behavior.

6.5.Integration Testing

Integration testing is a crucial phase in software development that focuses on combining individual units or modules and testing them as a group. This process helps identify defects in the interactions between modules, ensuring they communicate correctly, and function together as intended. By detecting interface issues, logical errors, and potential data mismatches, integration testing confirms that the application operates smoothly in a real-world environment. It ultimately enhances software reliability before proceeding to further testing stage.

Integration Testing 1: Login as Admin

Objective: To ensure that the admin panel integrates correctly with various modules like user, society, and event management.

Table 6.6: Summary of Admin Feature

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
--------------	----------------	----------------------------	-----------------	---------------

IT-001	Admin logs in and creates a new society	Login Module, Society Management Module	Admin successfully logs in and creates a new society	Pass
IT-002	Admin monitors metrics	Analytics Dashboard	Analytics of active user are displayed	Pass
IT-003	Admin enables/disables user's features access	Transcription Module, Summarization Module	User cannot process transcription and summarization requests	Pass
IT-004	Admin logs out	Admin Dashboard, Logout Module	Admin is logged out successfully	Pass

The integration testing table presents a series of test cases designed to verify how different modules of the Urdu Flow system interact and function together. Test Case IT-001 confirms that the Admin can successfully log in through the Login Module and, once authenticated, proceed to Admin account demonstrating proper communication between authentication and administrative components. Test Case IT-002 evaluates the system's analytics and monitoring capabilities by checking whether the platform accurately updates and displays the active user request rate, ensuring that the monitoring module correctly integrates with real-time usage data. Test Case IT-003 examines the Admin's ability to enable or disable user access to key features such as the transcription and summarization modules, and the system correctly prevents users from processing these requests when access is restricted, confirming that permission controls are properly linked across modules. Finally, Test Case IT-004 validates the logout process, ensuring that when the Admin initiates the logout action from the dashboard, the Logout Module terminates the session successfully and exits the system as expected. All integration test cases passed, indicating that the interconnected components of Urdu Flow operate smoothly and reliably together.

Integration Testing 2: Login as a user

Objective: To ensure that the correct screen and modules load for a user, and all features work as expected.

Table 6.7: Login as a User

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
IT-001	A user logs in and uploads a file	Login Module, File upload Module	File is successfully upload and visible to user	Pass
IT-002	User submits translation request	File upload module, Translation module, OCR module	Translation is served based on the source language into the target language	Pass
IT-003	User submits transcription request	File Upload Module, Transcription Module	The content of the file uploaded is provided to the user	Pass
IT-004	User requests summarization	File upload module, OCR module, summarization module	User is provided with the text summarization of the provided content	Pass
IT-005	User logs out	User Dashboard, Logout Module	User is logged out successfully	Pass

The integration testing table highlights the end-to-end interactions between multiple modules of the Urdu Flow application to ensure they operate cohesively during real user workflows. Test Case IT-001 verifies that a user can successfully log in through the Login Module and immediately proceed to upload a document using the File Upload Module, with the uploaded file becoming visible to the user as expected. Test Case IT-002 evaluates the translation workflow, where an uploaded file passes through the File Upload Module, OCR Module, and Translation Module, ensuring that the system accurately detects the source language and produces the correct translation in the chosen target language. Test Case IT-003 examines the transcription process by confirming that files uploaded through the File Upload Module are correctly handled by the Transcription Module, allowing the system to display the extracted

content to the user. Test Case IT-004 assesses the summarization pipeline, ensuring that uploaded or OCR-processed content flows properly into the Summarization Module so the user receives a clear, concise text summary of the provided material. Finally, Test Case IT-005 checks that the logout operation functions correctly, validating that the Logout Module successfully ends the user session and returns the user to the login interface. All integration test cases passed, demonstrating that the modules within Urdu Flow communicate effectively and deliver the expected outcomes during combined operations.

6.6. Tools and Technologies

Table 6.8: Tools and Technologies for Proposed Projects

Tools And Technologies	Tools	Version	Rationale
	Android Studio	2024.1.1	Flutter Based Project
	PyCharm	2024.3	Python IDE
	Firebase	Latest	Backend services (Auth, DB, Storage)
	MS PowerPoint	365	Presentation Creation
	MS Word	365	Project Documentation
	MS Project	365	Gantt Chart
	Google Colab		Running Python Training Scripts
	Google Cloud Platform		Cloud Service Provider
	Technology	Version	Rationale
	Dart	2.1	Programming Language
	Python	3.12.0	Programming Language
	Flutter	3.22	Platform
	Hugging Face Transformers	4.33.0	Natural Language Processing
	Google Translate API		Speech Recognition

6.7. Summary

This chapter presented the different types of testing conducted on the Urdu Flow application, including unit testing, functional testing, and integration testing. Core modules such as user authentication, file upload and processing, translation, transcription, and summarization were thoroughly examined using both standard and boundary-input scenarios. Each test case was executed by comparing the system's actual behavior against the expected outcomes to ensure accuracy and reliability across all features. Successful and edge-case executions were documented, and the final status of each test case was marked as Pass or Fail based on how the

system responded. The testing process confirmed that the major components of Urdu Flow perform correctly and interact smoothly within the complete workflow of the application.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7. Conclusion

The Urdu Flow application has been developed as a bilingual mobile solution aimed at simplifying language processing tasks between Urdu and English. Based on the objectives outlined in the Project Proposal, SRS, and SDD, the system integrates core functionalities including speech-to-text transcription, text translation, document summarization, and file processing. These features collectively address the growing need for accurate and accessible Urdu-English language assistance for students, educators, professionals, and everyday users.

Throughout the development phase, each module such as user authentication, file upload and management, transcription, translation, and summarization was designed with usability, performance, and reliability in mind. All core components were tested against expected outcomes, confirming that the system operates as intended. The mobile interface has been kept user-friendly and straightforward, enabling users with varying technical backgrounds to easily interact with the application without requiring prior training.

Urdu Flow addresses a clear gap in existing tools by offering a unified platform for processing Urdu and English content. The application reduces dependency on multiple third-party apps, especially for users who frequently deal with bilingual content, scanned documents, or audio-based information. The integration of modern NLP and machine learning models ensures acceptable accuracy for transcription and summarization, while the use of the Google Translate API provides dependable translation outputs enhanced by additional refinement steps. In summary, the Urdu Flow application successfully meets the core requirements defined at the start of the project. It provides an accessible, organized, and practical solution for speech recognition, translation, transcription, and summarization, fulfilling its goal of communication and content processing in a convenient mobile environment.

7.1. Future Work

Although the current version of Urdu Flow performs effectively and aligns with the defined objectives, there are several opportunities to enhance the system in future iterations. These improvements can broaden functionality, strengthen performance, and expand accessibility for a wider range of users. Some possible areas for future work include:

7.1.1. Offline Processing Capabilities

Implement selective **offline transcription, translation, or summarization** using lightweight on-device models, allowing users to process content without relying on an internet connection.

7.1.2. Advanced AI and Model Fine-Tuning

Integrate additional fine-tuned NLP models for:

- More accurate regional Urdu accent recognition.
- Improved summarization for long documents.
- Better context-aware translations beyond standard API outputs.

7.1.3. Web-Based Admin and User Dashboard

Introduce a web dashboard for managing user sessions, viewing analytics, and reviewing system logs. This would streamline administrative tasks and allow easier oversight of system performance.

7.1.4. Enhanced OCR and Document Handling

Improve OCR accuracy for handwritten Urdu text, scanned documents, and low-resolution images through custom-trained vision models.

CHAPTER 8

REFERENCE

Reference

- [1] J. Ashraf, N. Iqbal, N. S. Khattak and A. M. Zaidi, “Speaker Independent Urdu speech recognition using HMM,” *2010 The 7th International Conference on Informatics and Systems (INFOS)*, Cairo, Egypt, 2010, pp. 1–5, doi: 10.1007/978-3-642-13881-2_14.
- [2] K. Ahmad *et al.*, “Challenges in Urdu Machine Translation,” in *Proc. 2024 Conf. on Low-Resource Machine Translation*, 2024, pp. 25–32. [Online]. Available: <https://aclanthology.org/2024.loresmt-1.4.pdf>
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Proc. 31st Conf. on Neural Information Processing Systems (NeurIPS)*, Long Beach, CA, USA, 2017, pp. 5998–6008
- [4] U. Mohy ud Din, “Urdu-English Machine Transliteration using Neural Networks,” *arXiv preprint arXiv:2001.05296*, 2020. doi: 10.48550/arXiv.2001.05296.
- [5] Fiaz, L., Tahir, M. H., Shams, S., & Hussain, S. (2025). UrduLLaMA 1.0: Dataset curation, preprocessing, and evaluation in low-resource settings. *arXiv*. <https://doi.org/10.48550/arXiv.2502.16961>
- [6] Google AI Blog. "Advances in Speech Recognition for Low-Resource Languages." Google Research, 2023, <https://ai.googleblog.com/2023/05/advances-in-speech-recognition.html>
- [7] Khan, A., and S. Hussain. "A Comprehensive Survey of Named Entity Recognition for Urdu." *International Journal of Advanced Computer Science and Applications*, Vol. 9, No. 6, 2018, <http://www.ijacsa.thesai.org/pdf/v9i6/28.pdf>.
- [8] Chopra, R., et al. "Abstractive Summarization of Long Documents using Pointer-Generator Networks." *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017 <https://aclanthology.org/P17-1023>
- [9] H. Israr, M. K. Shahzad, and S. Anwar, “Improved Urdu–English Neural Machine Translation with a fully convolutional neural network encoder,” *International Journal of Mathematical, Engineering and Management Sciences*, vol. 9, no. 5, pp. 1067–1088, 2024, doi: 10.33889/IJMEMS.2024.9.5.056.
- [10] Basit, A., Azeemi, A. H., & Raza, A. A. (2024). *Challenges in Urdu Machine Translation*. In *Proceedings of the Seventh Workshop on Technologies for Machine Translation of Low-Resource Languages (LoResMT 2024)* (pp. 44–49). Association for Computational Linguistics, <https://aclanthology.org/2024.loresmt-1.4.pdf>